MATRIX MULTIPLICATION ON GPU

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated Intelligent Systems Lab.

CUDA PERFORMANCE CONSIDERATIONS

- Setting number of threads
  - Get the max #threads per SM
- Branch divergence
  - Causes under-utilization. Avoid it
- Shared memory
  - Capture the locality with manually managed cache-like memory
  - Avoid bank conflict
- Coalescing
  - Align memory accesses to maximize bandwidth
- Overlap IO with computation
  - With 'stream's

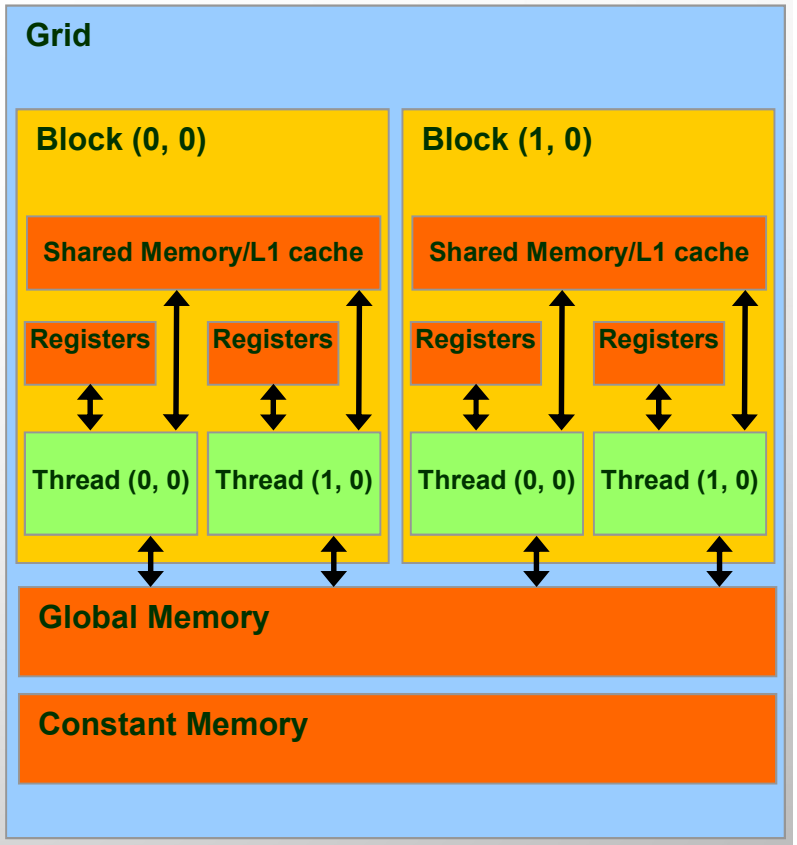
CONTENT

- Matrix multiplication
  - Shared memory
  - Coalescing
- Matrix transpose
  - Bank conflict

MATMUL KERNEL

CUDA MEMORY MODEL

- CUDA memories
  - Read/write per-thread
    registers (~1 cycle)
  - Read/write per-block
    shared memory (~5 cycles)
  - Read/write per-grid
    global (off-chip) memory (~500 cycles)
  - Read/only per-grid
    constant memory (~5 cycles with caching)



*This is a conceptual diagram.
Not the hardware model

CUDA VARIABLE TYPE QUALIFIERS

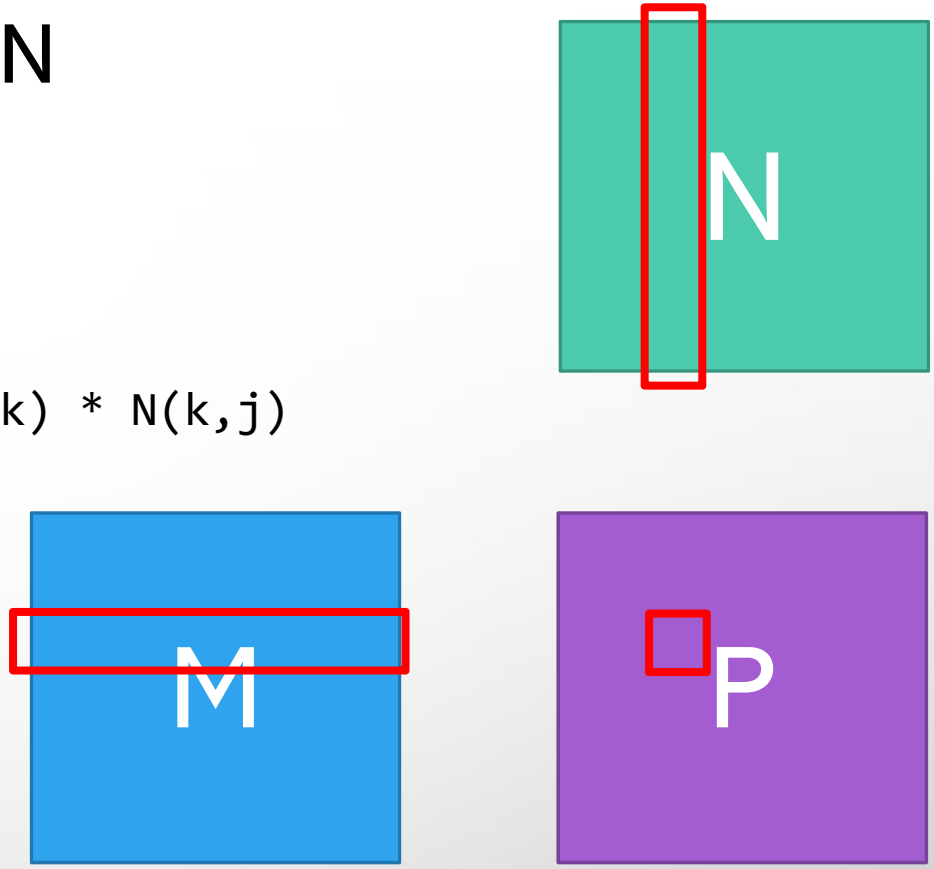
Variable declaration	Memory	Scope	Lifetime
`int LocalVar;`	register	thread	thread
`__device__ __shared__ int SharedVar;`	shared	block	block
`__device__ int GlobalVar;`	global	grid	application
`__device__ __constant__ int ConstantVar;`	constant	grid	application

- `__device__` is optional when it's trivial
  - Used with `__shared__`, or `__constant__`

- (Automatic) variables without any qualifier reside in a register
  - Except per-thread arrays that reside in global memory

MATRIX MULTIPLICATION

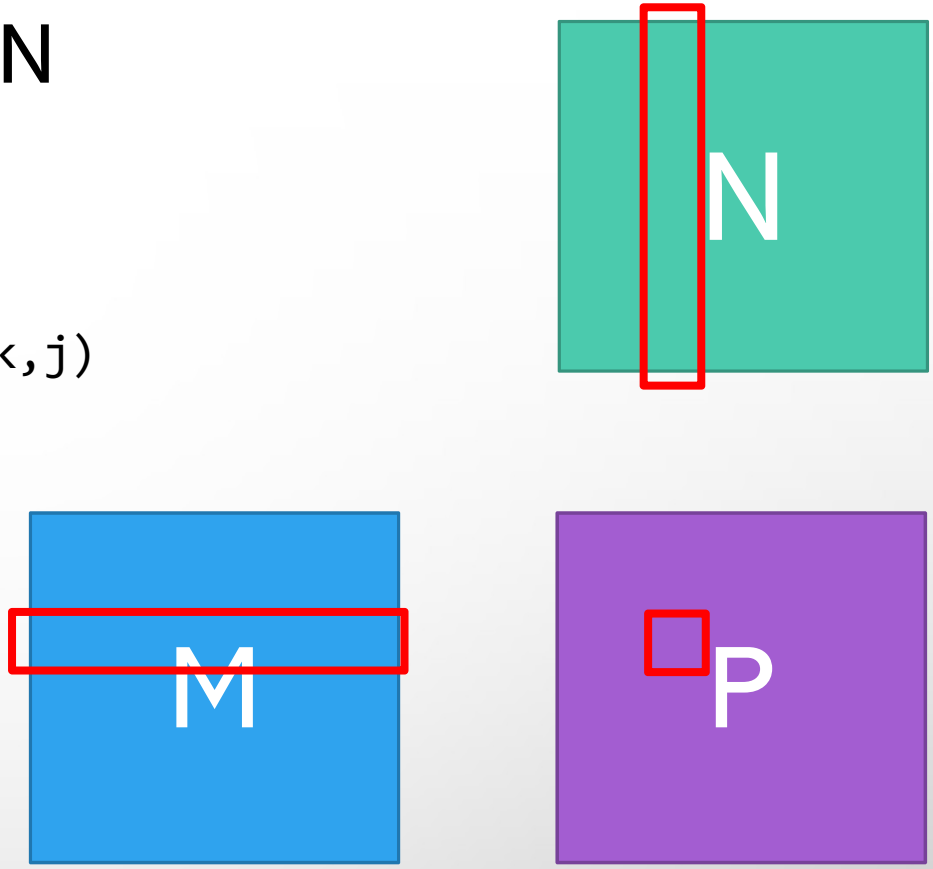
```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            P(i,j) = P(i,j) + M(i,k) * N(k,j)
```



- Let's revisit this with CUDA

MATRIX MULTIPLICATION

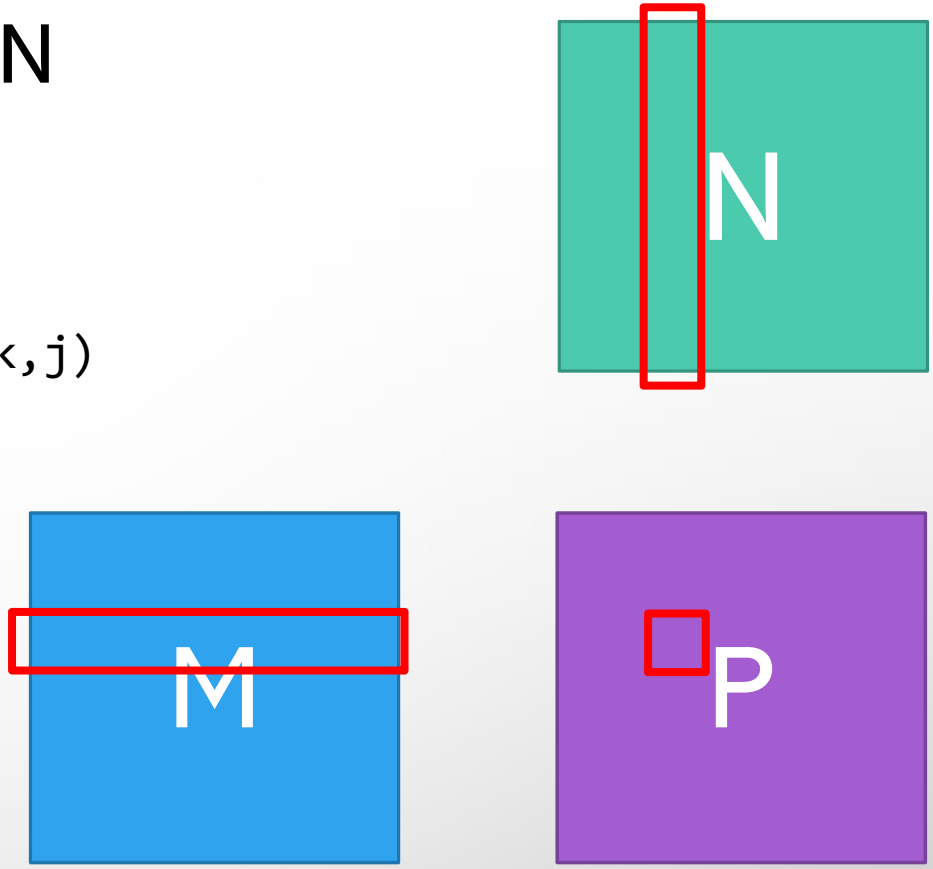
```
Get i,j from block/threadIdx
for k = 1 to n
    P(i,j) = P(i,j) + M(i,k) * N(k,j)
```



- It's rational to let each thread create one P(i,j) element

MATRIX MULTIPLICATION

```
Get i,j from block/threadIdx
for k = 1 to n
    P(i,j) = P(i,j) + M(i,k) * N(k,j)
```



- This causes frequent global memory access
  - Wasting bandwidth and latency

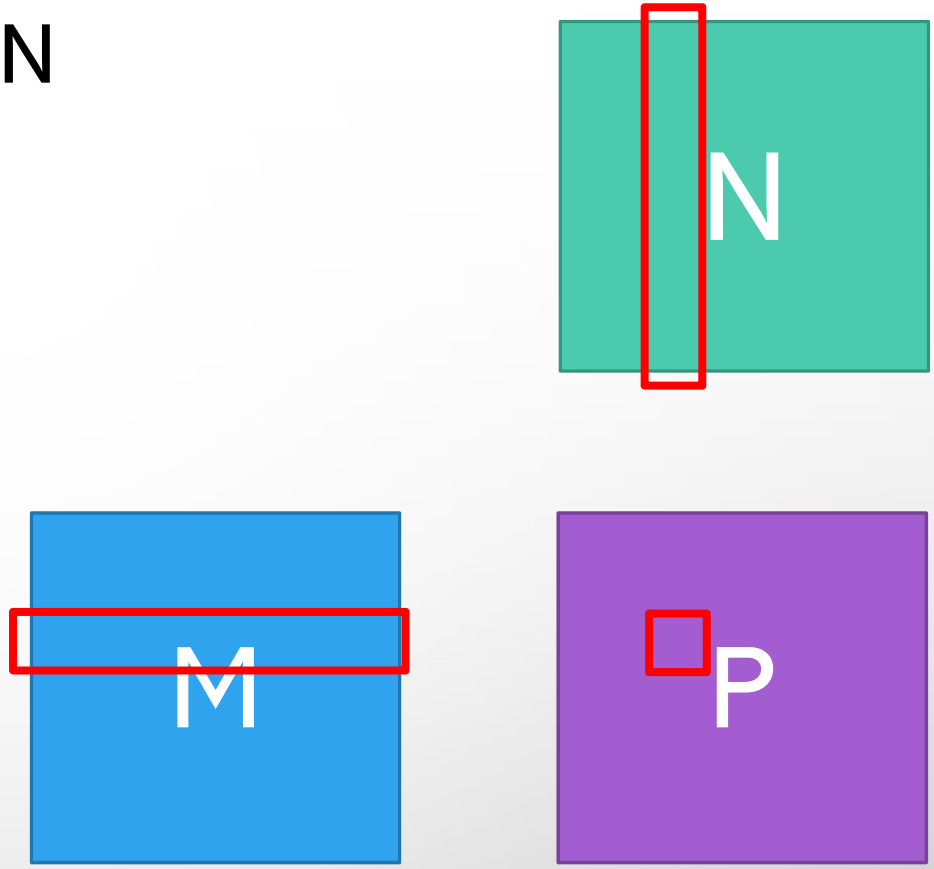
- In CPU, cache could reduce this. What about GPUs?

THREAD SCHEDULING

- Jetson Orin Nano has 28~192KB L1 cache
- An SM can run up to 1536 threads
  - Depending on architecture
- Cache per thread? 19~128B
  - This is barely a cacheline
- GPU cache can capture spatial locality, but usually not temporal locality
- +) L1 cache has 128B lines
  - To support 32 threads in a warp * 4B

MATRIX MULTIPLICATION

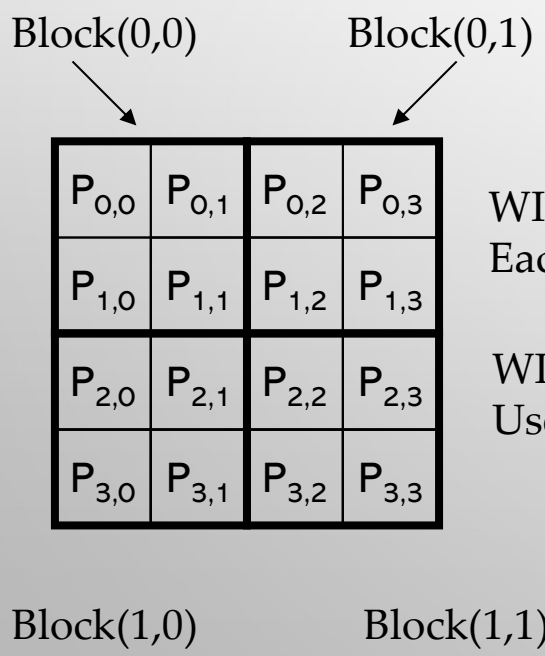
```
Get i,j from block/threadIdx
sum = 0
for k = 1 to n
    sum += M(i,k) * N(k,j)
P(i,j) = sum
```



- Use a local variable (register)
- More registers, less memory accesses

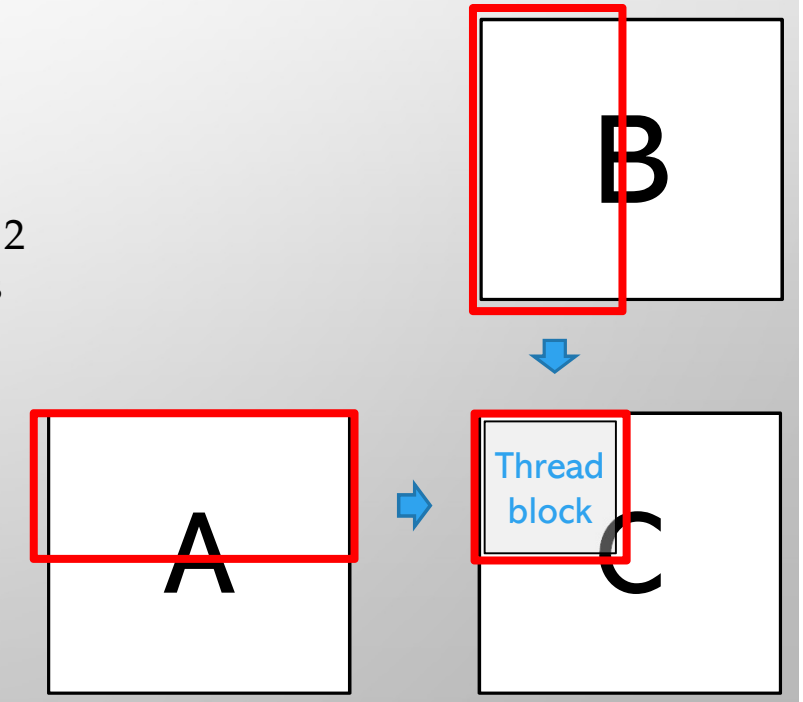
DEFINING THREADBLOCKS

- We will reuse the 2Dblock strategy
- Have each 2D thread block to compute a $(TILE_WIDTH)^2$ sub-matrix (tile) of the result matrix
  - Each has $(TILE_WIDTH)^2$ threads
- Generate a 2D Grid of $(WIDTH/TILE_WIDTH)^2$ blocks
  - Of course, we want to make each threadblock larger than a warp size (32)



WIDTH = 4; TILE_WIDTH = 2
Each block has 2*2 = 4 threads

WIDTH/TILE_WIDTH = 2
Use 2*2 = 4 blocks



KERNEL INVOCATION

```
// Setup the execution configuration
// Assume square Matrix
// TILE_WIDTH is a #define constant

   dim3 dimGrid(Width/TILE_WIDTH, Width/TILE_WIDTH, 1);
   dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);


// Launch the device computation threads
MatrixMulKernel<<<dimGrid, dimBlock>>>(M_d, N_d, P_d, Width);
```

A SIMPLE MATRIX MULTIPLICATION KERNEL

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  float value=0;
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  int k = 0;
  for(k = 0; k < width; ++k) {
    value += M[row * width + k] * N[k * width + col];
  }
  P[row * width + col] = value; //write the accumulated result
}
```

WORK FOR BLOCK (0,0) (TILEWIDTH=2)

blockDim.x

blockDim.y

Col = 0 * 2 + threadIdx.x

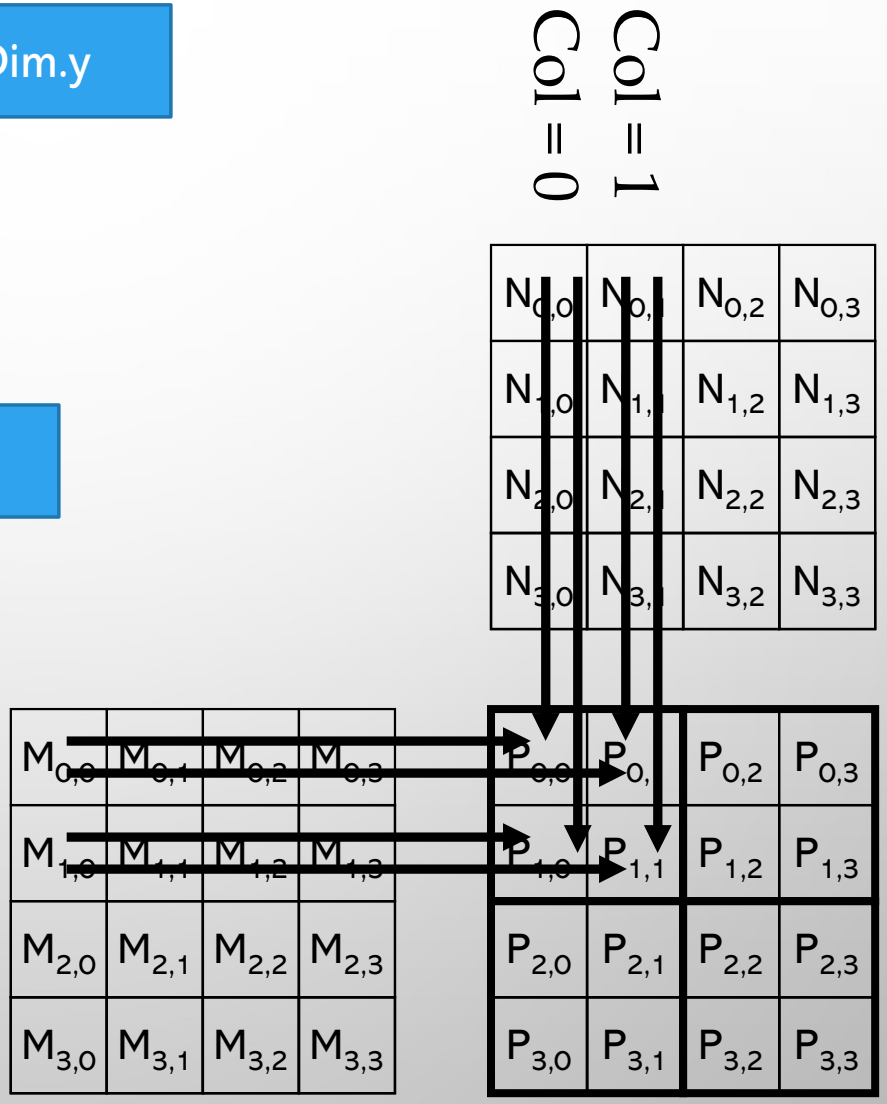
Row = 0 * 2 + threadIdx.y

blockIdx.x

blockIdx.y

Row = 0

Row = 1



WORK FOR BLOCK (0,1)

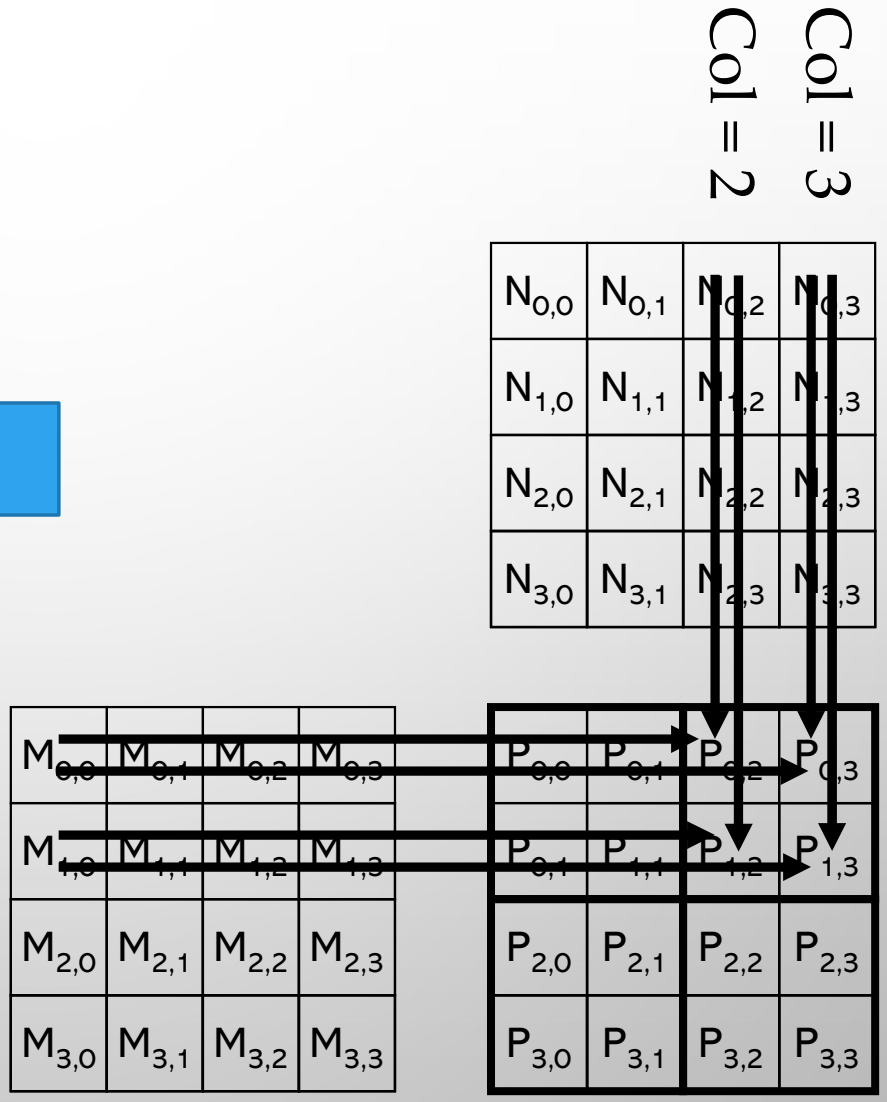
Col = 1 * 2 + threadIdx.x
Row = 0 * 2 + threadIdx.y

blockIdx.x

blockIdx.y

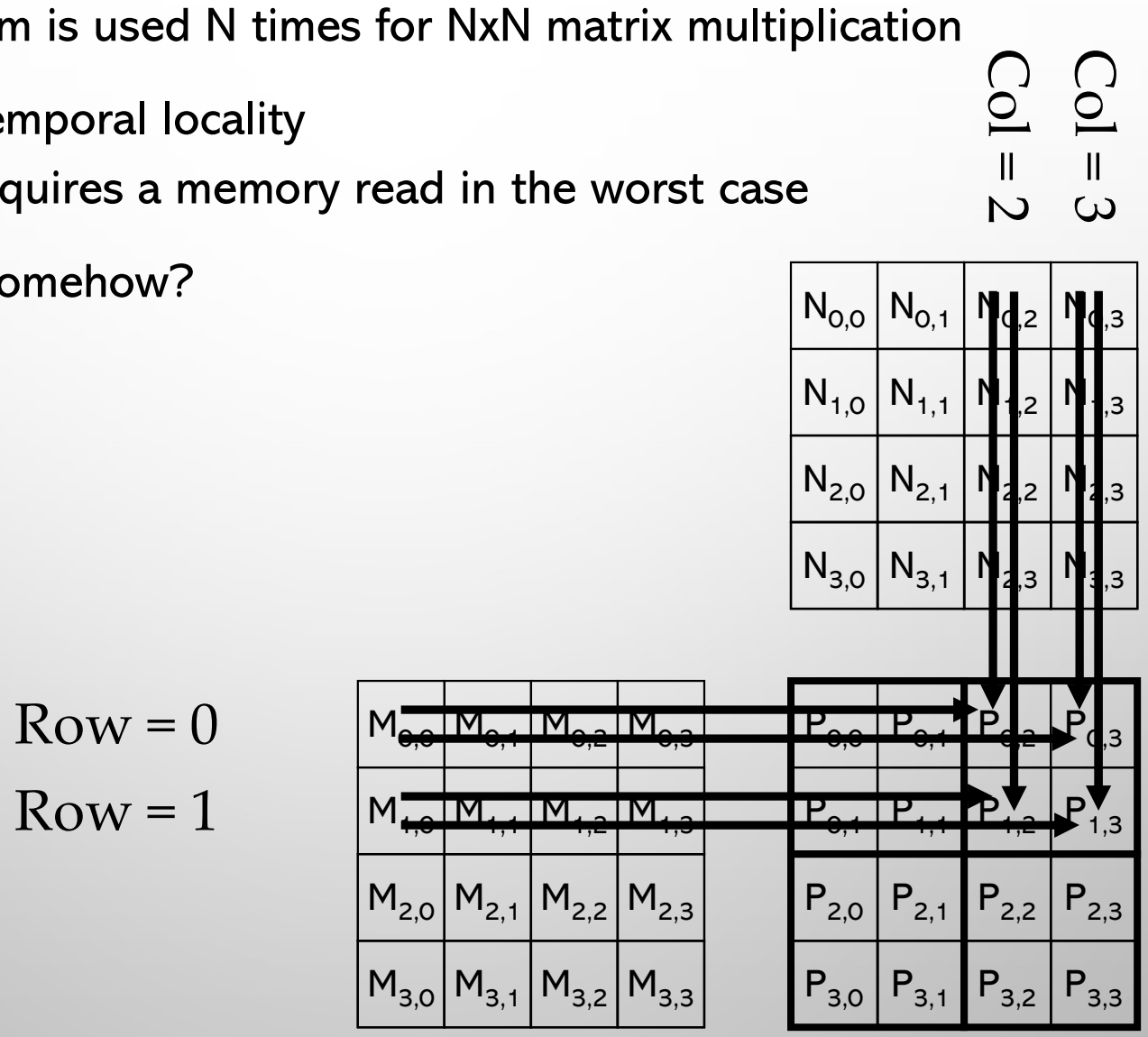
Row = 0

Row = 1



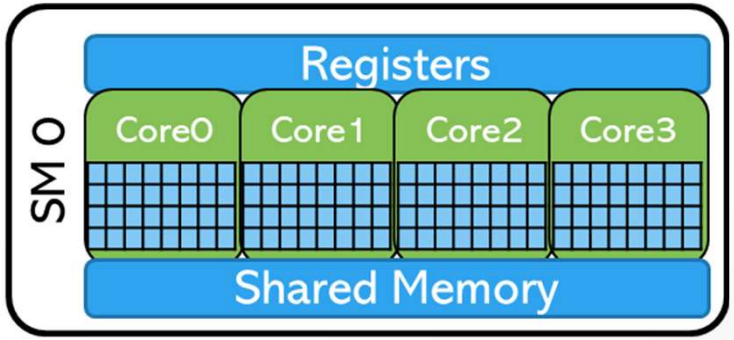
MEMORY ACCESS PROBLEM

- Problem: each item is used N times for NxN matrix multiplication
- Small L1 → No temporal locality
  → Each access requires a memory read in the worst case
- Can we cache it somehow?



USING SHARED MEMORY

SHARED MEMORY



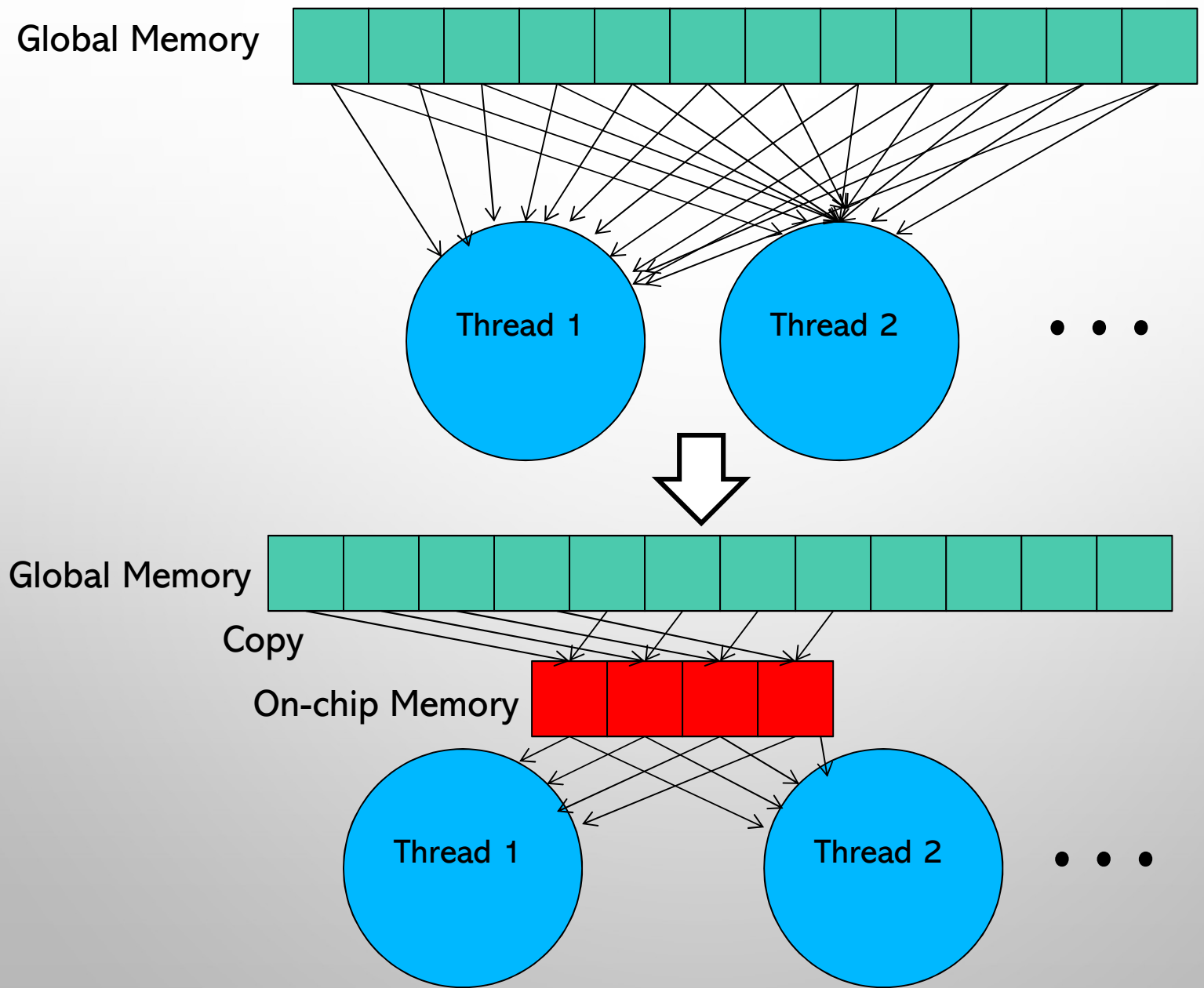
- Shared memory has two important features

1. It is shared within a threadblock
2. It has low latency, high bandwidth

- max 164KB shared memory per SM
  - Is it better than L1 cache?
  - Yes, shared memory is <u>manually managed</u>
  - And <u>shared among threads</u>
  - A kind of scratchpad memory

Variable declaration		Memory	Scope	Lifetime
	`int LocalVar;`	register	thread	thread
`__device__ __shared__`	`int SharedVar;`	shared	block	block
`__device__`	`int GlobalVar;`	global	grid	application
`__device__ __constant__`	`int ConstantVar;`	constant	grid	application

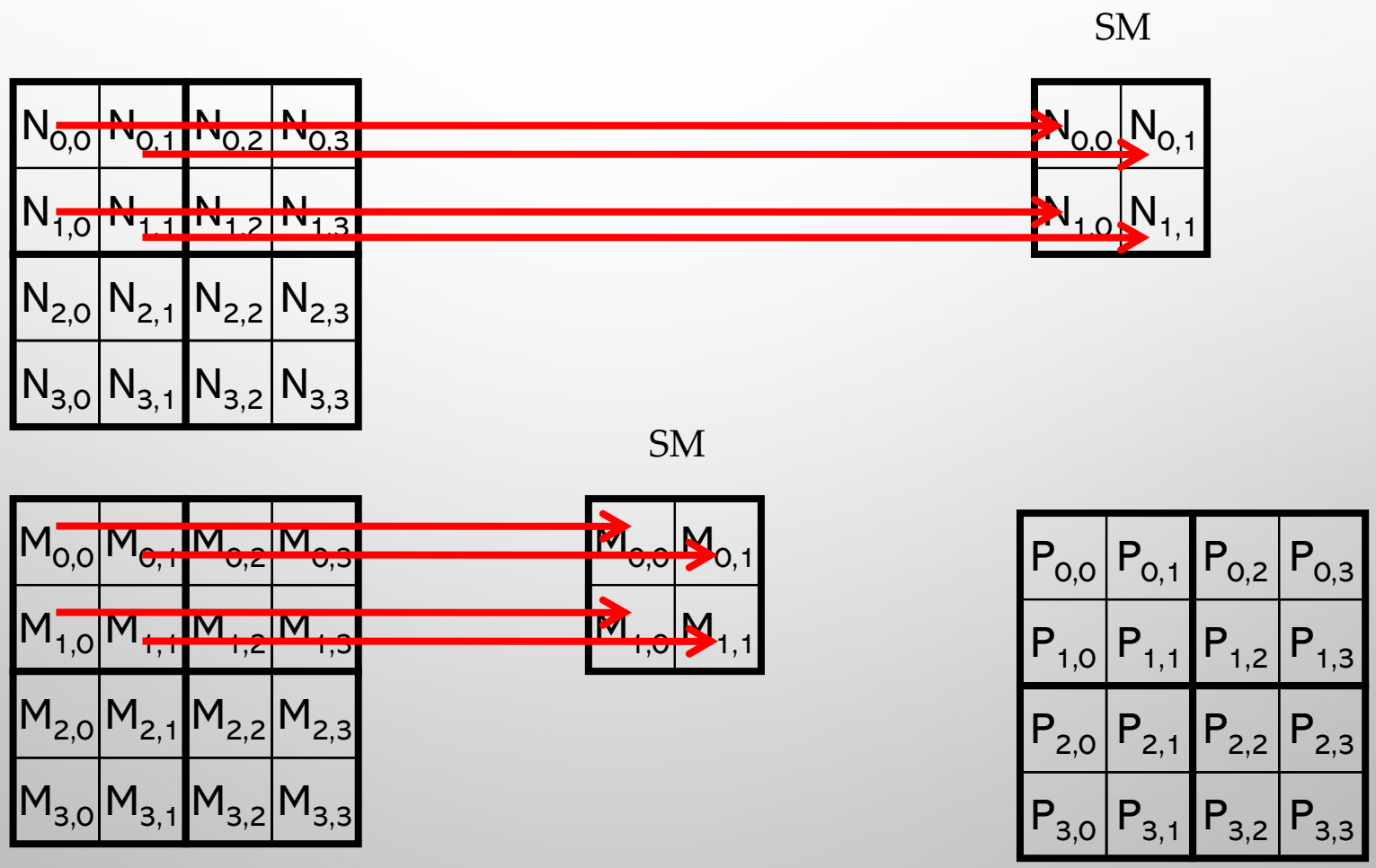
SHARED MEMORY BLOCKING BASIC IDEA



PROGRAMMING WITH SHARED MEMORY

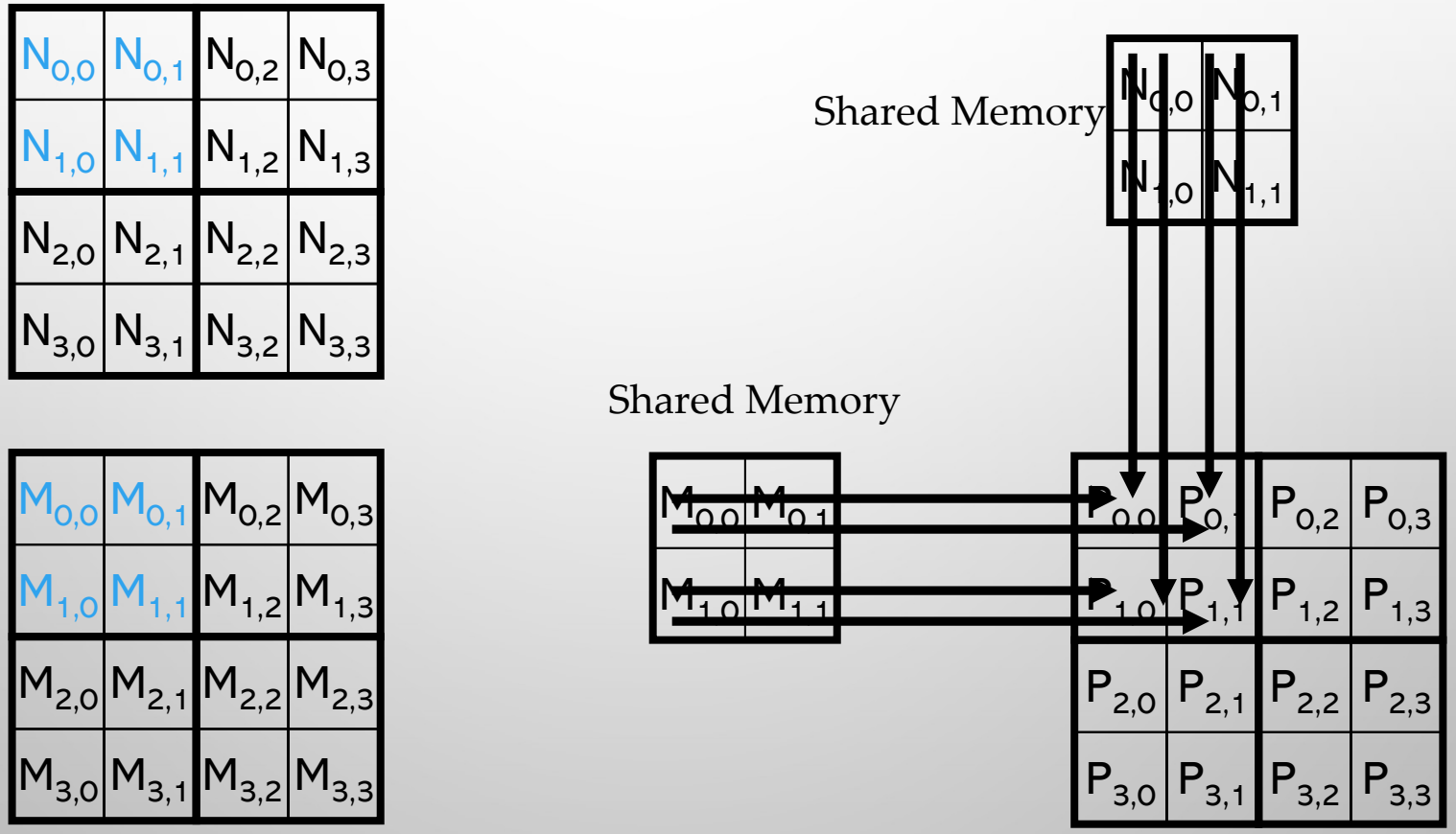
```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];
…
```

WORK FOR BLOCK (0,0)



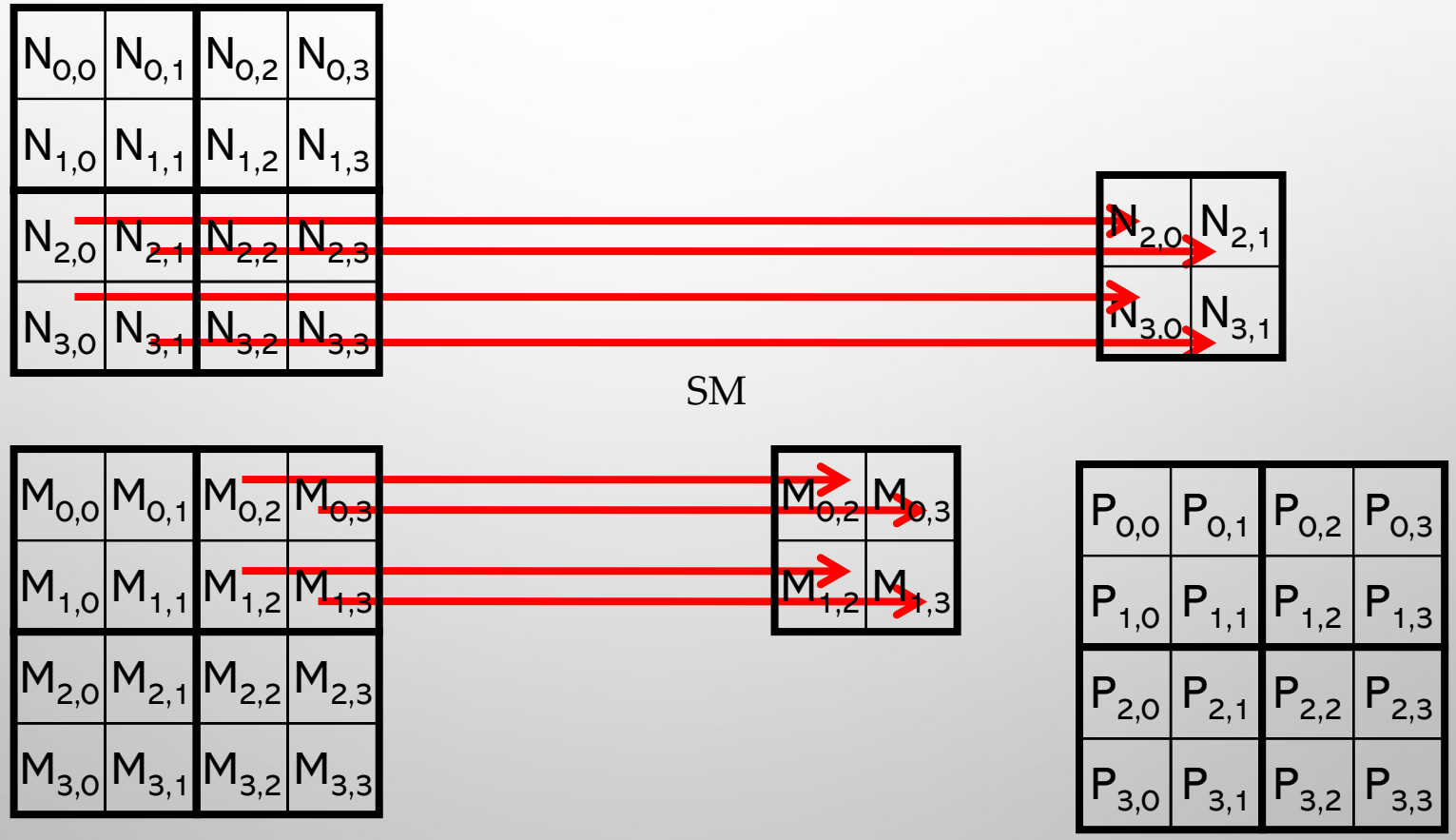
Make sure the copy is done

WORK FOR BLOCK (0,0)



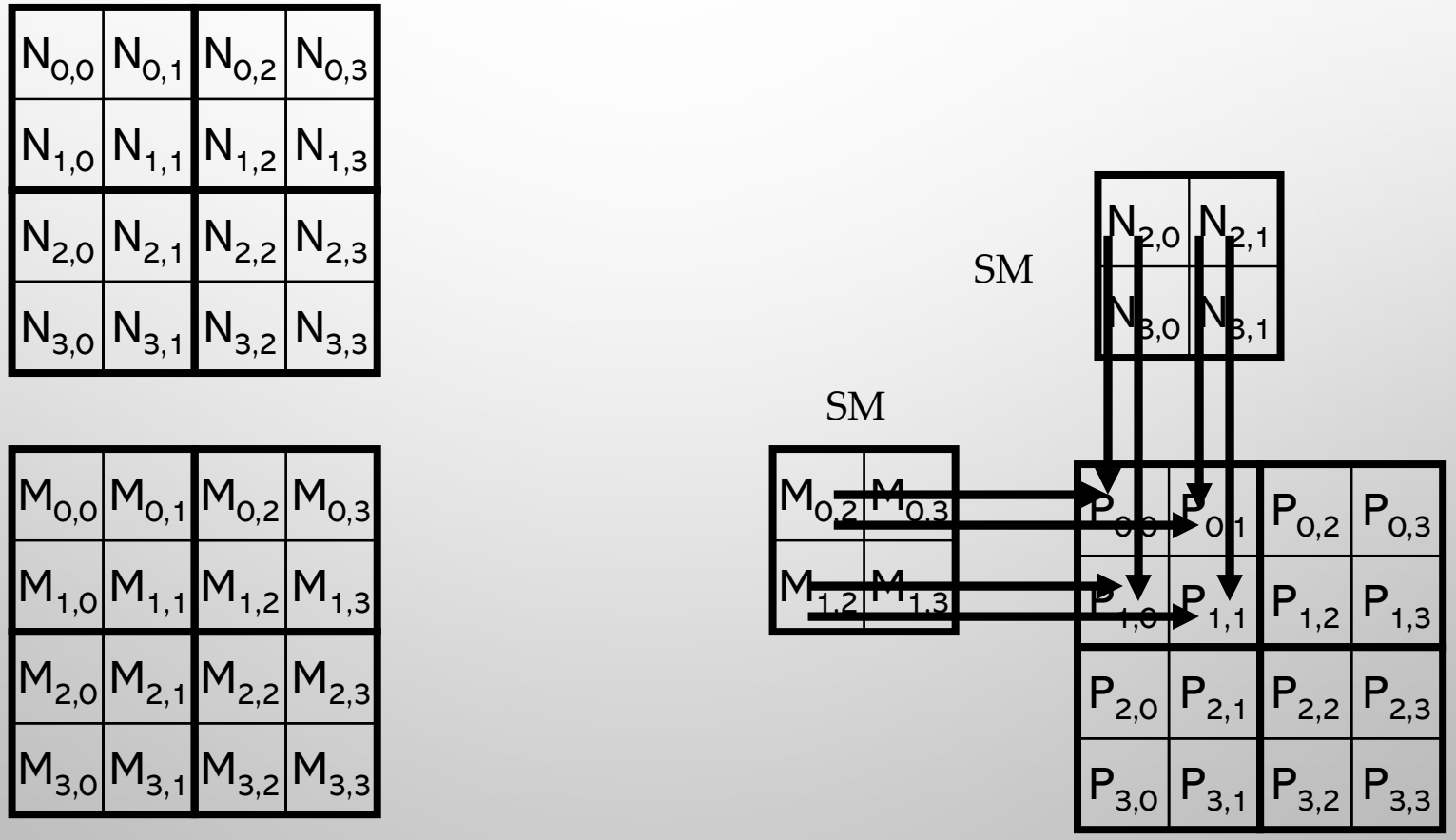
Make sure the computation is done

WORK FOR BLOCK (0,0)



Make sure the copy is done

WORK FOR BLOCK (0,0)



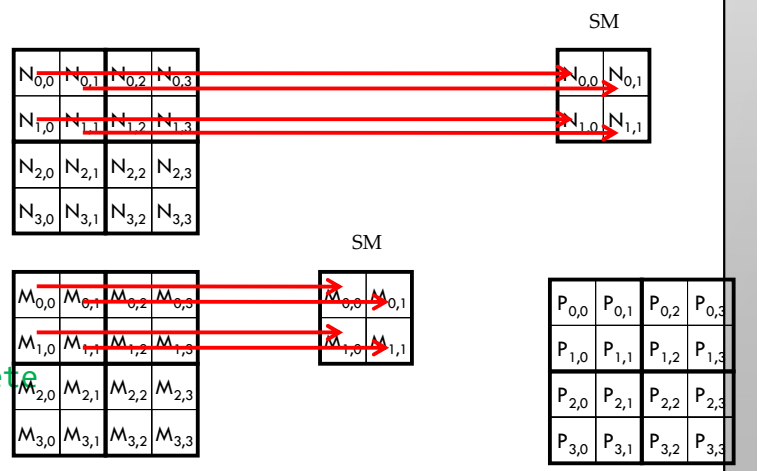
Make sure the computation is done

TILED MATRIX MULTIPLICATION KERNEL

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
1.  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
2.  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

3.  int bx = blockIdx.x;  int by = blockIdx.y;
4.  int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
5.  int Row = by * TILE_WIDTH + ty;
6.  int Col = bx * TILE_WIDTH + tx;
7.  float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
8.  for (int m = 0; m < Width/TILE_WIDTH; ++m) {
       // Load shared memory phase
9.       subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
10.      subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
          __syncthreads(); // Wait for the copy to finish
       // Compute phase
12.    for (int k = 0; k < TILE_WIDTH; ++k)
13.          Pvalue += subTileM[ty][k] * subTileN[k][tx];
14.      __syncthreads(); // Wait for the compute to be complete
15. }
16. P[Row*Width+Col] = Pvalue;
}
```

TILED MATRIX MULTIPLICATION KERNEL

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
1.  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
2.  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

3.  int bx = blockIdx.x;  int by = blockIdx.y;
4.  int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
5.  int Row = by * TILE_WIDTH + ty;
6.  int Col = bx * TILE_WIDTH + tx;
7.  float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
8.  for (int m = 0; m < Width/TILE_WIDTH; ++m) {
      // Load shared memory phase
9.      subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
10.     subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
         __syncthreads(); // Wait for the copy to finish
      // Compute phase
12.    for (int k = 0; k < TILE_WIDTH; ++k)
13.          Pvalue += subTileM[ty][k] * subTileN[k][tx];
14.     __syncthreads(); // Wait for the compute to be complete
15. }
16. P[Row*Width+Col] = Pvalue;
}
```

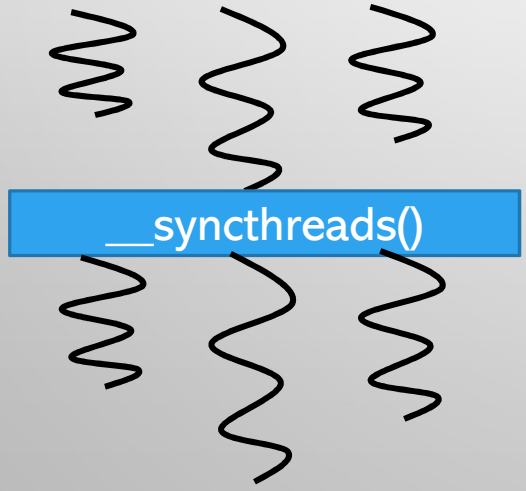


__SYNCTHREADS()

- __syncthreads() is a barrier synchronization **within the threadblock**

- All threads in the same block must reach the __syncthreads() before anyone can move on

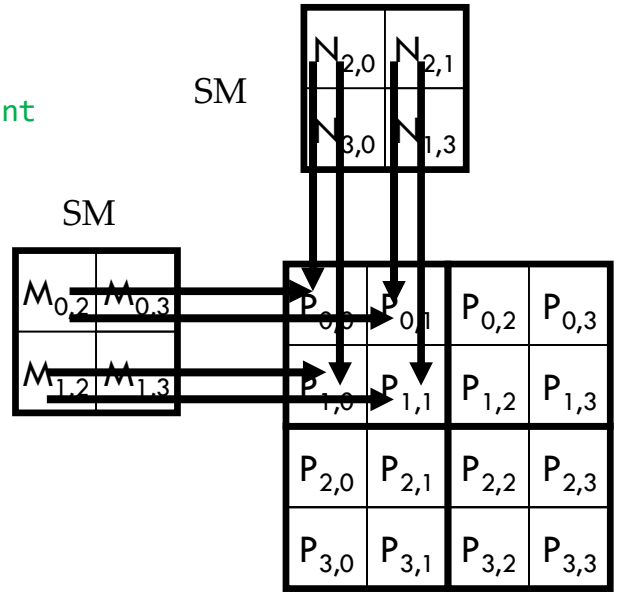
- Best used to coordinate tiled algorithms
  - To ensure that all elements of a tile are loaded
  - To ensure that all elements of a tile are consumed



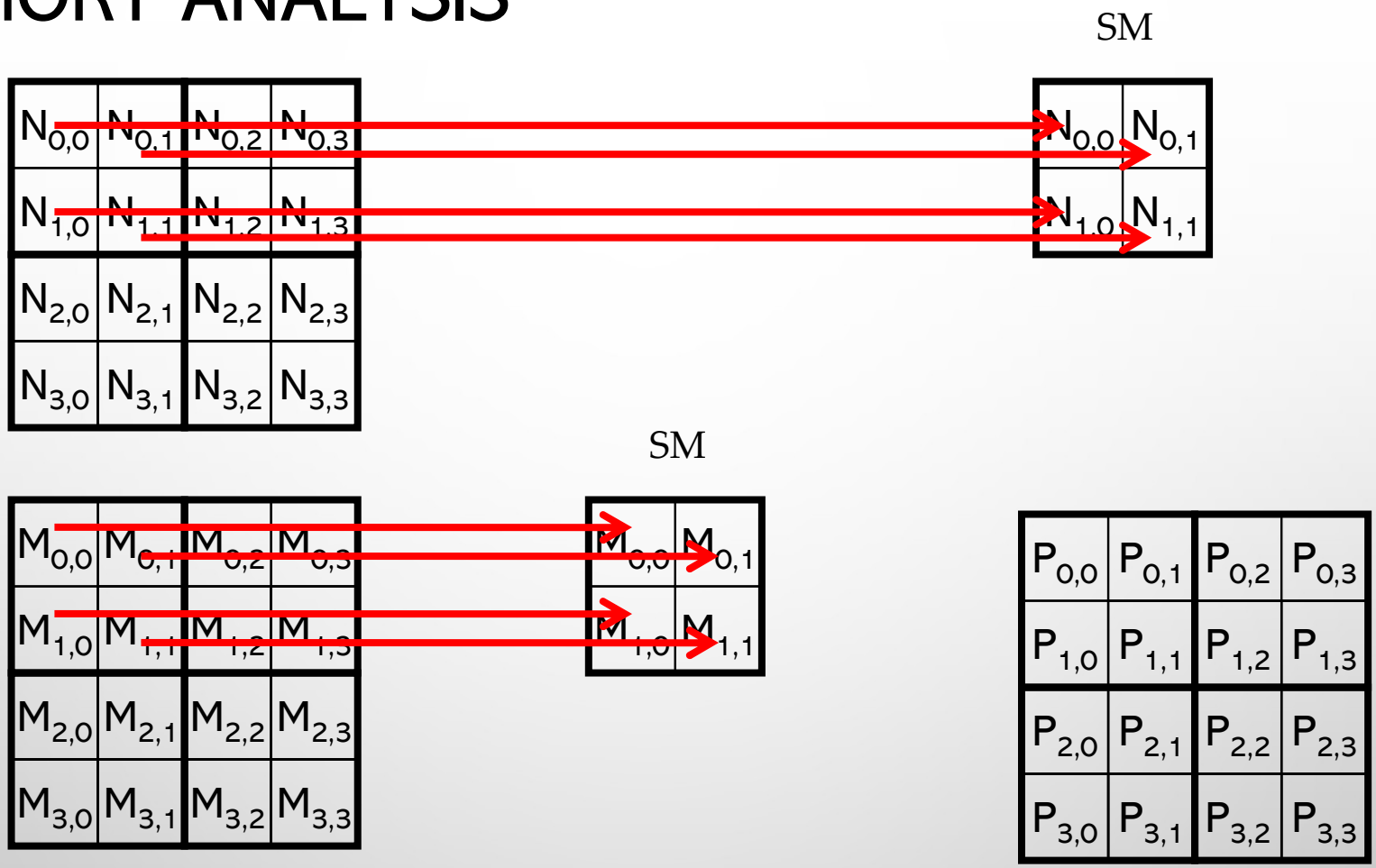
Needs the entire threadblock to push it away

TILED MATRIX MULTIPLICATION KERNEL

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
1.  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
2.  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

3.  int bx = blockIdx.x;  int by = blockIdx.y;
4.  int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
5.  int Row = by * TILE_WIDTH + ty;
6.  int Col = bx * TILE_WIDTH + tx;
7.  float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
8.  for (int m = 0; m < Width/TILE_WIDTH; ++m) {
       // Load shared memory phase
9.       subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
10.      subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
         __syncthreads(); // Wait for the copy to finish
       // Compute phase
12.    for (int k = 0; k < TILE_WIDTH; ++k)
13.          Pvalue += subTileM[ty][k] * subTileN[k][tx];
14.      __syncthreads(); // Wait for the compute to be complete
15. }
16. P[Row*Width+Col] = Pvalue;
}
```

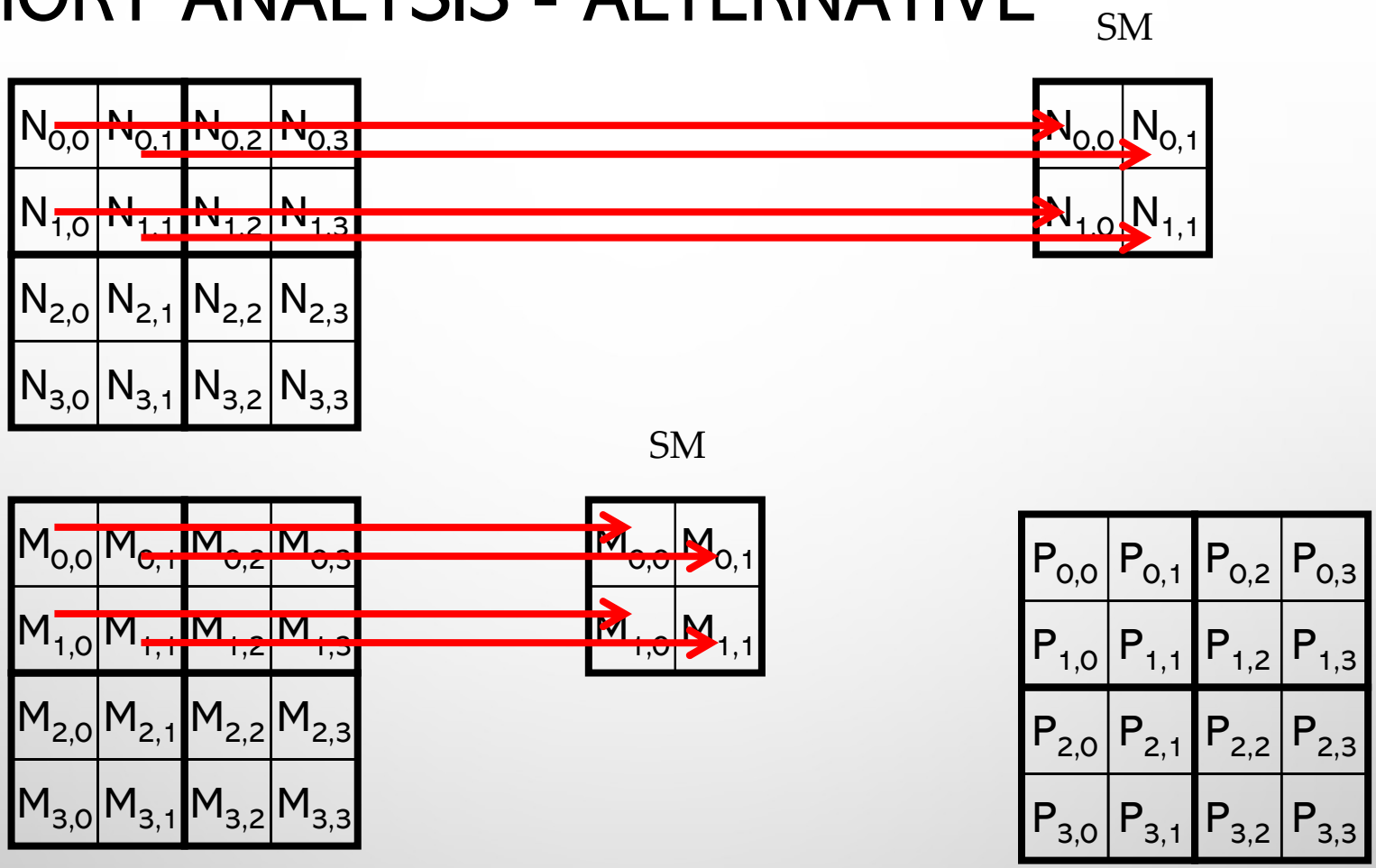


MEMORY ANALYSIS



- Each element in the tile is reused b times (b: block size)
  - Speedup: b
  - Larger b, larger speedup

MEMORY ANALYSIS - ALTERNATIVE



- Each threadblock loads a 'bxb' tile N/b times
- There are $(N/b)^2$ threadblocks
- Total N^3/b loads
  - b is the speedup

HOW LARGE OF A TILE?

- Shared memory size = 164KB
- Shared memory per Matrix: 82KB
- Block size = 82KB / 4B =20992
- Sqrt(20992) ~= 145
- So 145x145 block?

1. 145 x 145 is larger than the max threadblock size (1024)
2. 145 x 145 is not a multiple of 32

HOW LARGE OF A TILE?

- 8x8 block = 64 threads = 2 warps
- shared memory usage
  - 8x8x2x4B = 512B per threadblock
- Max threads per SM
  - Limited by max blocks
  - 64*16 = 1024 threads (67% occupancy)
- Total shared memory per SM: 512*16 = 8KB (too small)

- Max #threads: 1536
- Max warps: 48
- Max thread blocks: 16
- Registers: 65536
- Shared memory: 164KB

HOW LARGE OF A TILE?

- 16x16 block = 256 threads = 8 warps
- shared memory usage
  - 16x16x2x4B = 2048B per threadblock
- Max threads per SM
  - Limited by max threads

  256*6 = 1536 threads (100% occupancy)
- Total shared memory per SM: 2048*6 = 12KB (too small)

- Max #threads: 1536
- Max warps: 48
- Max thread blocks: 16
- Registers: 65536
- Shared memory: 164KB

HOW LARGE OF A TILE?

- 32x32 block = 1024 threads = 32 warps
- shared memory usage
  - 32x32x2x4B = 8192B per threadblock
- Max threads per SM
  - Limited by max threads

  1024*1 = 1024 threads (67% occupancy)
- Total shared memory per SM: 8192*1 = 8KB (too small)

- Max #threads: 1536
- Max warps: 48
- Max thread blocks: 16
- Registers: 65536
- Shared memory: 164KB

HOW LARGE OF A TILE?

- 32x32 block = 1024 threads = 32 warps
- shared memory usage
  - 32x32x2x4B = 8192B per threadblock
- Max threads per SM
  - Limited by max threads

  1024*1 = 1024 threads (67% occupancy)
- Total shared memory per SM: 8192*1 = 8KB (too small)

- Max #threads: 1536
- Max warps: 48
- Max thread blocks: 16
- Registers: 65536
- Shared memory: 164KB

- In this example, we trade-off between blocksize and cuda occupancy
  - The only way to find out is to measure
  - My guess: larger blocksize is better. 100% occupancy is not necessary to achieve memory bandwidth

HOW LARGE OF A TILE?

- 32x32 block = 1024 threads = 32 warps
- shared memory usage
  - 32x32x2x4B = 8192B per threadblock
- Max threads per SM
  - Limited by max threads

  1024*1 = 1024 threads (67% occupancy)
- Total shared memory per SM: 8192*1 = 8KB (too small)

- Max #threads: 1536
- Max warps: 48
- Max thread blocks: 16
- Registers: 65536
- Shared memory: 164KB

- In this example, we trade-off between block (tile) size and cuda occupancy
  - The only way to find out is to measure
  - My guess: larger block (tile) size is better. 100% occupancy is not necessary to achieve memory bandwidth
- A lot of shared memory is being wasted
- We can increase tilesize while keeping the threadblocksize same by letting each thread handle multiple output elements (try it yourself in the Lab)

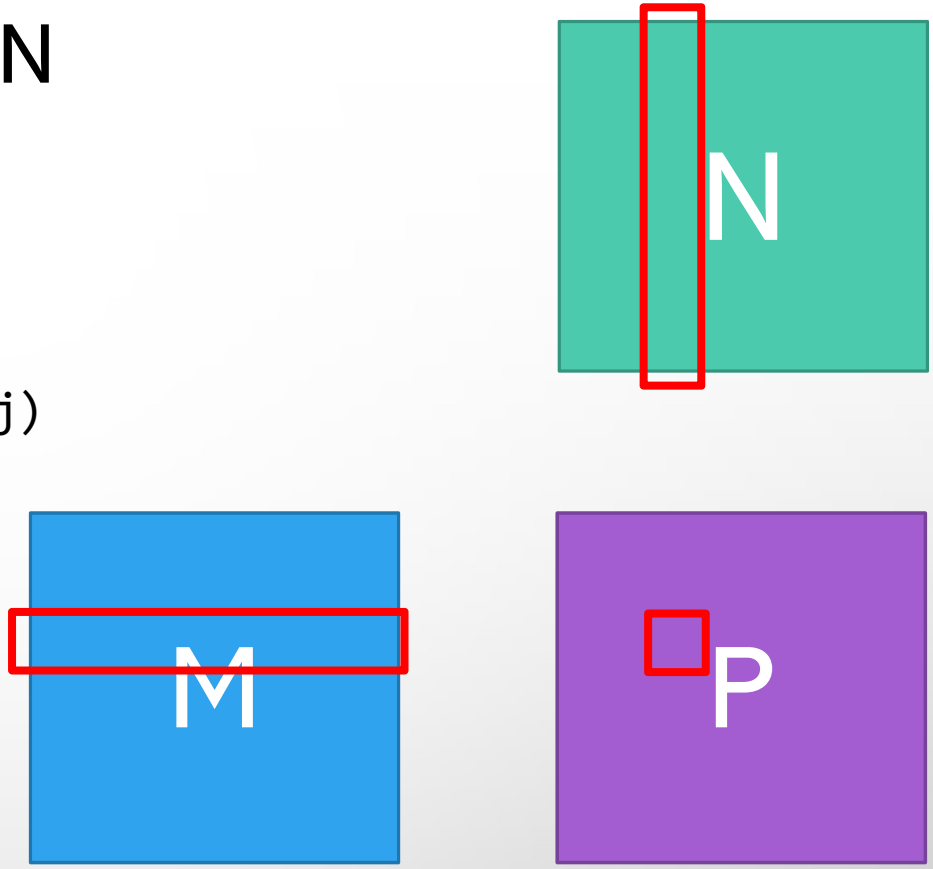
COALESCING

FLASHBACK – A SIMPLE MATMUL KERNEL

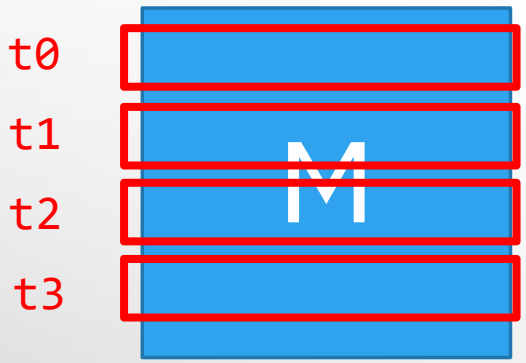
```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  float value=0;
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  int k = 0;
  for(k = 0; k < width; ++k) {
    value += M[row * width + k] * N[k * width + col];
  }
  P[row * width + col] = value; //write the accumulated result
}
```

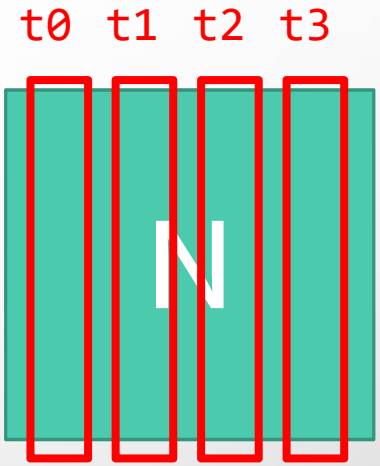
MATRIX MULTIPLICATION

```
Get i,j from block/threadIdx
sum = 0
for k = 1 to n
    sum += P(i,j) + M(i,k) * N(k,j)
P(i,j) = sum
```



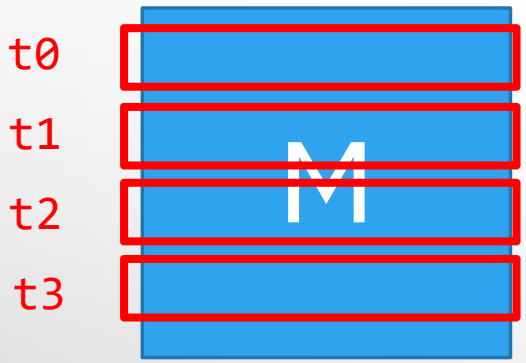
MATRIX MULTIPLICATION

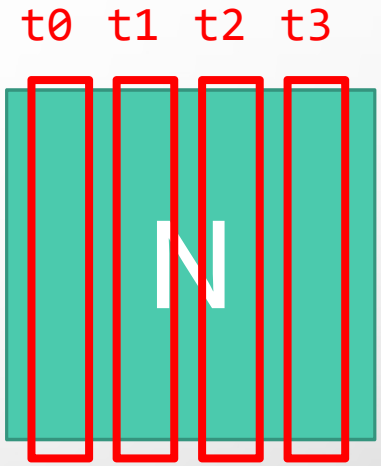




- Q: which one is faster?

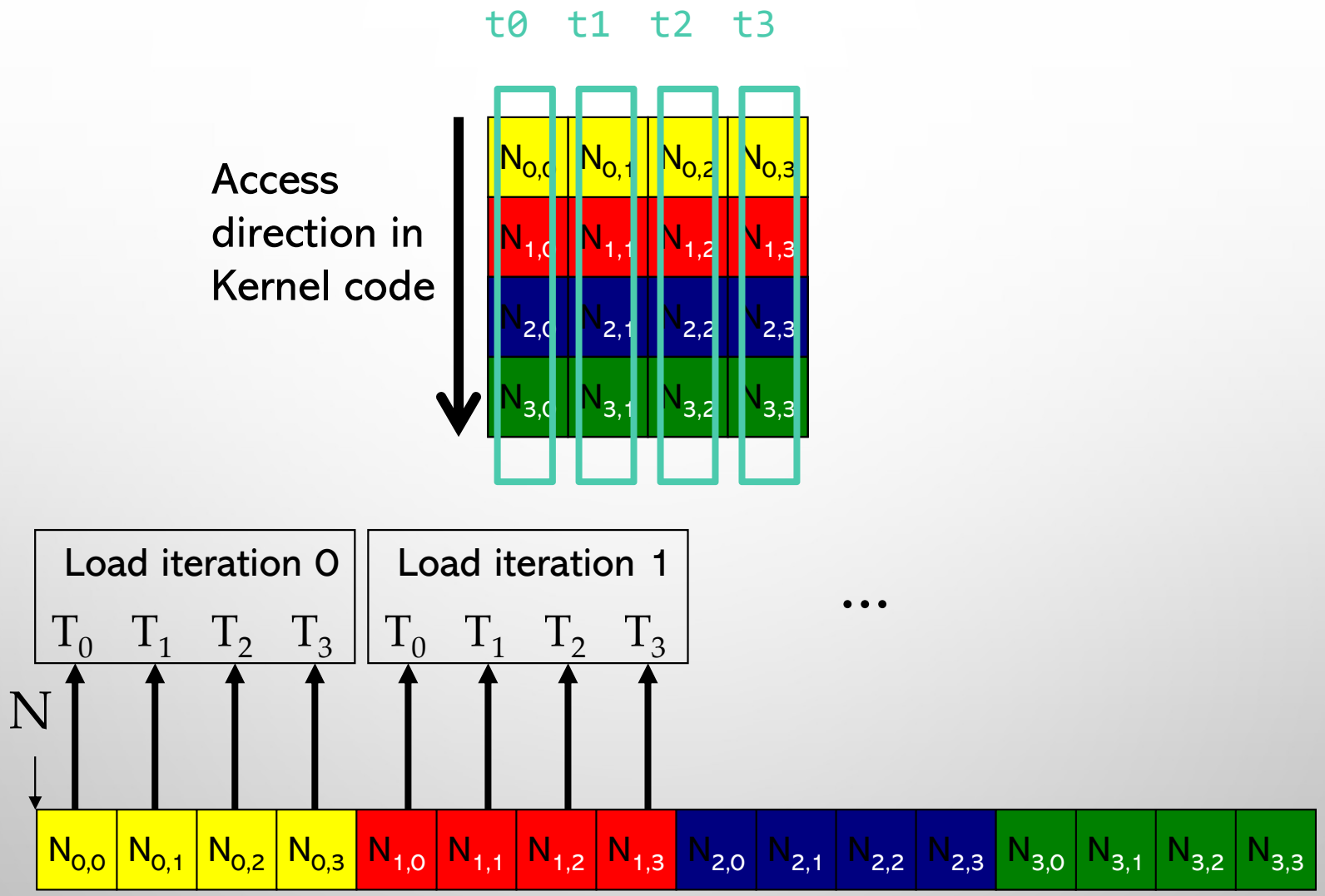
MATRIX MULTIPLICATION





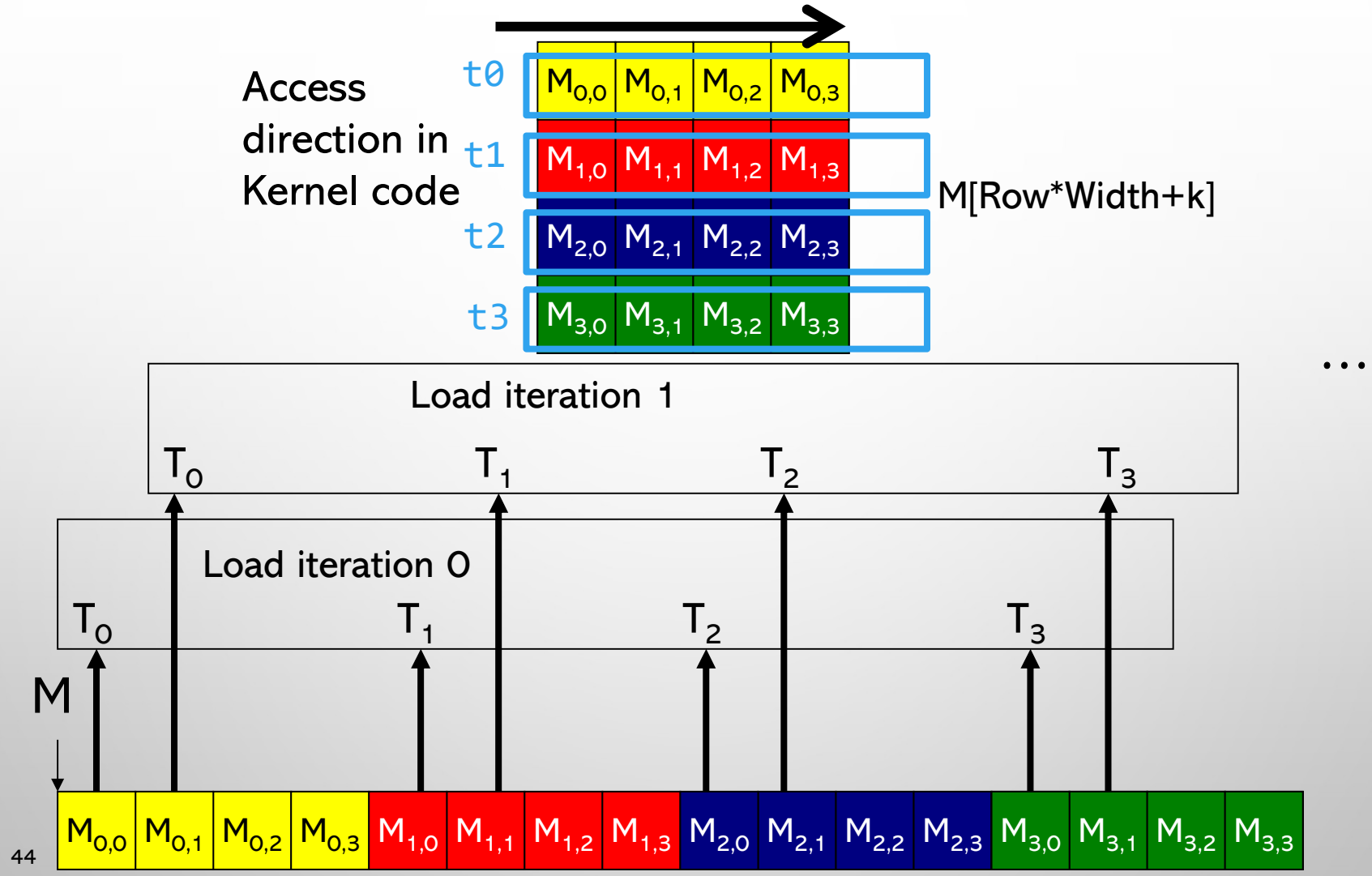
- Q: which one is faster?
- A: You're probably thinking M, but the answer is N

COALESCING



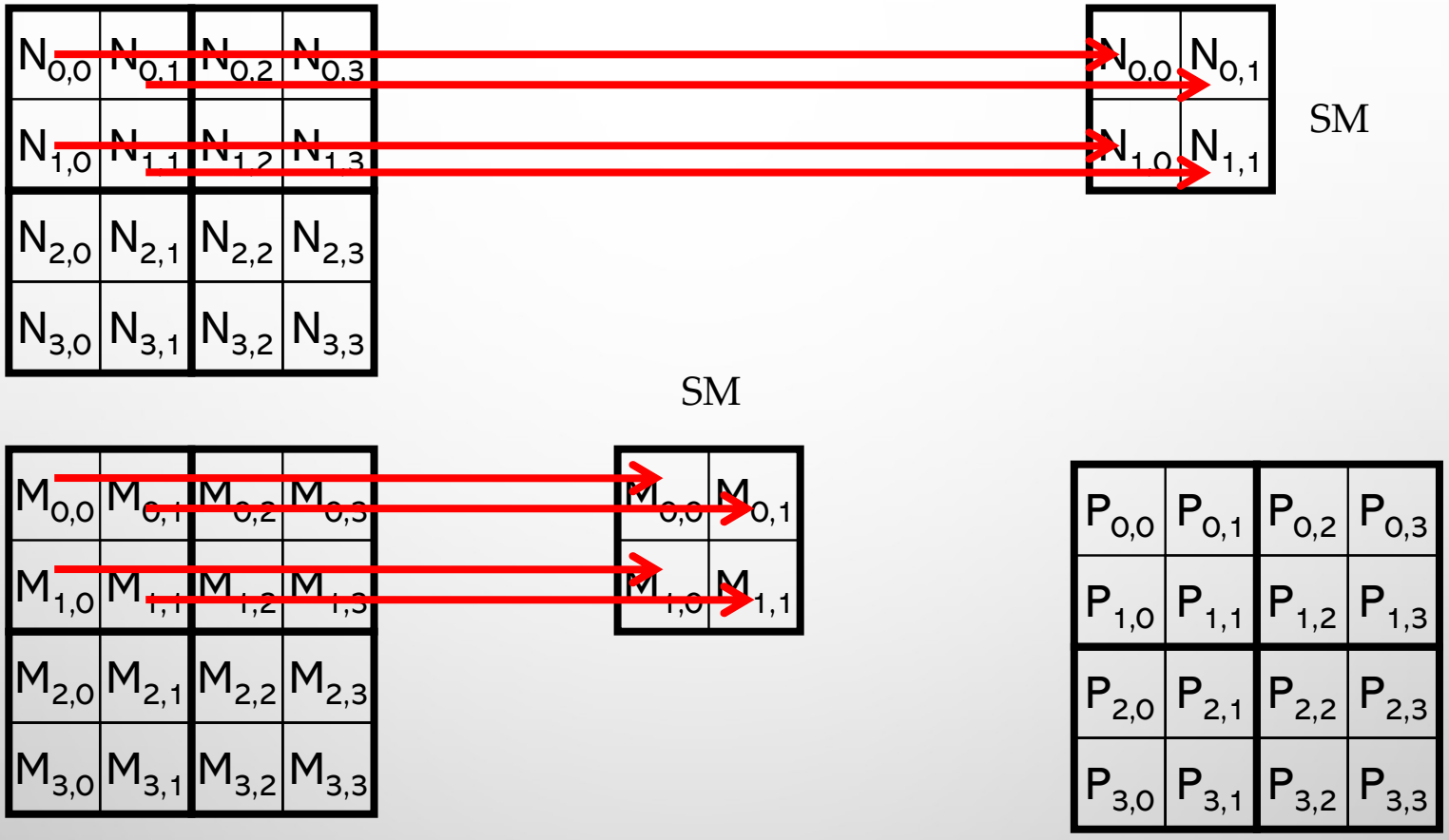
- When threads in a warp access the same cache line (128B), the accesses are <u>coalesced</u>
  - Meaning, only one access is made to the memory

NO COALESCING



- Slowdown from more memory accesses
- And the cache is too small

CORNER TURNING



- Even though the access to M is originally not coalesced, we can load an M tile into SM in a coalesced pattern
- Then read from SM instead of global memory

IS OUR KERNEL IN A COALESCED ORDER?

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
1.  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
2.  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

3.  int bx = blockIdx.x;  int by = blockIdx.y;
4.  int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
5.  int Row = by * TILE_WIDTH + ty;
6.  int Col = bx * TILE_WIDTH + tx;
7.  float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
8.  for (int m = 0; m < Width/TILE_WIDTH; ++m) {
       // Load shared memory phase
9.       subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
10.      subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
         __syncthreads(); // Wait for the copy to finish
       // Compute phase
12.    for (int k = 0; k < TILE_WIDTH; ++k)
13.          Pvalue += subTileM[ty][k] * subTileN[k][tx];
14.      __syncthreads(); // Wait for the compute to be complete
15. }
16. P[Row*Width+Col] = Pvalue;
}
```

MATRIX TRANSPOSE AND SHARED MEMORY BANK CONFLICTS

DYNAMIC SHARED MEMORY ALLOCATION

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
1.  __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
2.  __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];


…
```

- We have been using static shared memory allocation
  - We assign one element per thread
- What if we want to launch our kernel multiple times with different block sizes?

```
// Launch the kernel with multiple block sizes..

SomeKernel<<<N/128, 128>>>(parameters..);
SomeKernel<<<N/64, 64>>>(parameters..);
SomeKernel<<<N/32, 32>>>(parameters..);
```

DYNAMIC SHARED MEMORY ALLOCATION

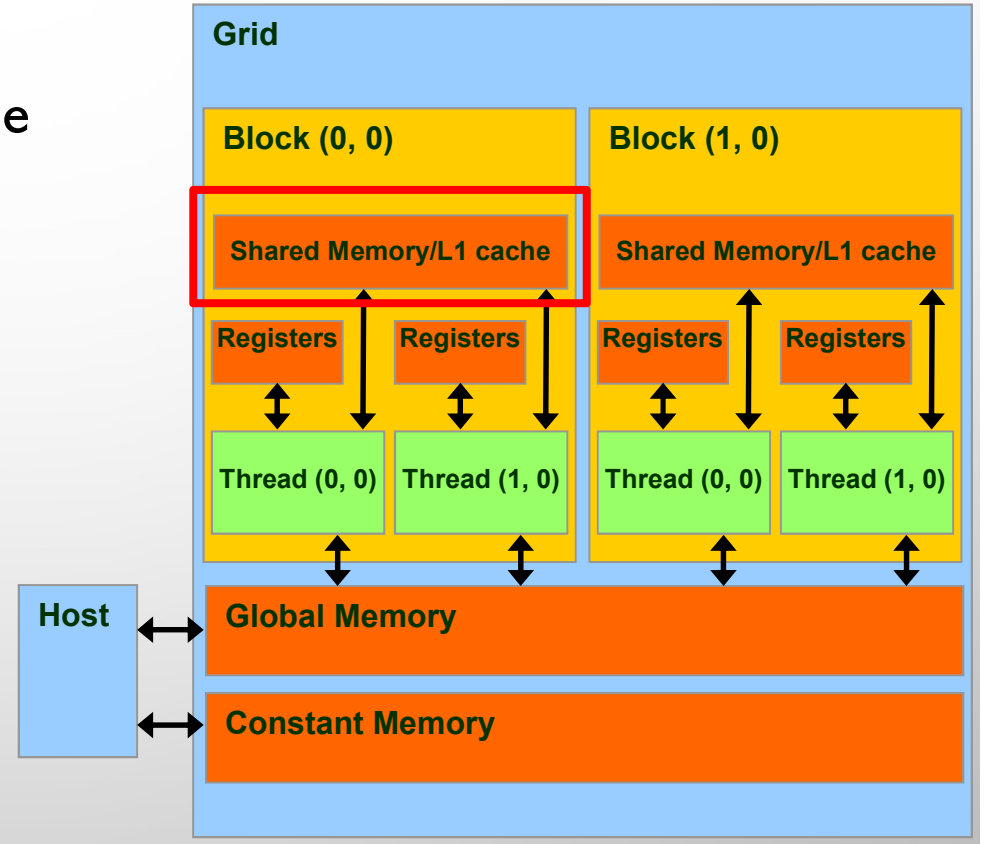
```
__global__ void someKernel(int* A)
{
   extern __shared__ int sm[];
```

```
    // Launch the kernel with multiple block sizes..

    SomeKernel<<<N/128, 128, 128*sizeof(float) >>> (parameters..);
    SomeKernel<<<N/64, 64, 64*sizeof(float) >>> (parameters..);
    SomeKernel<<<N/32, 32, 32*sizeof(float) >>> (parameters..);
```

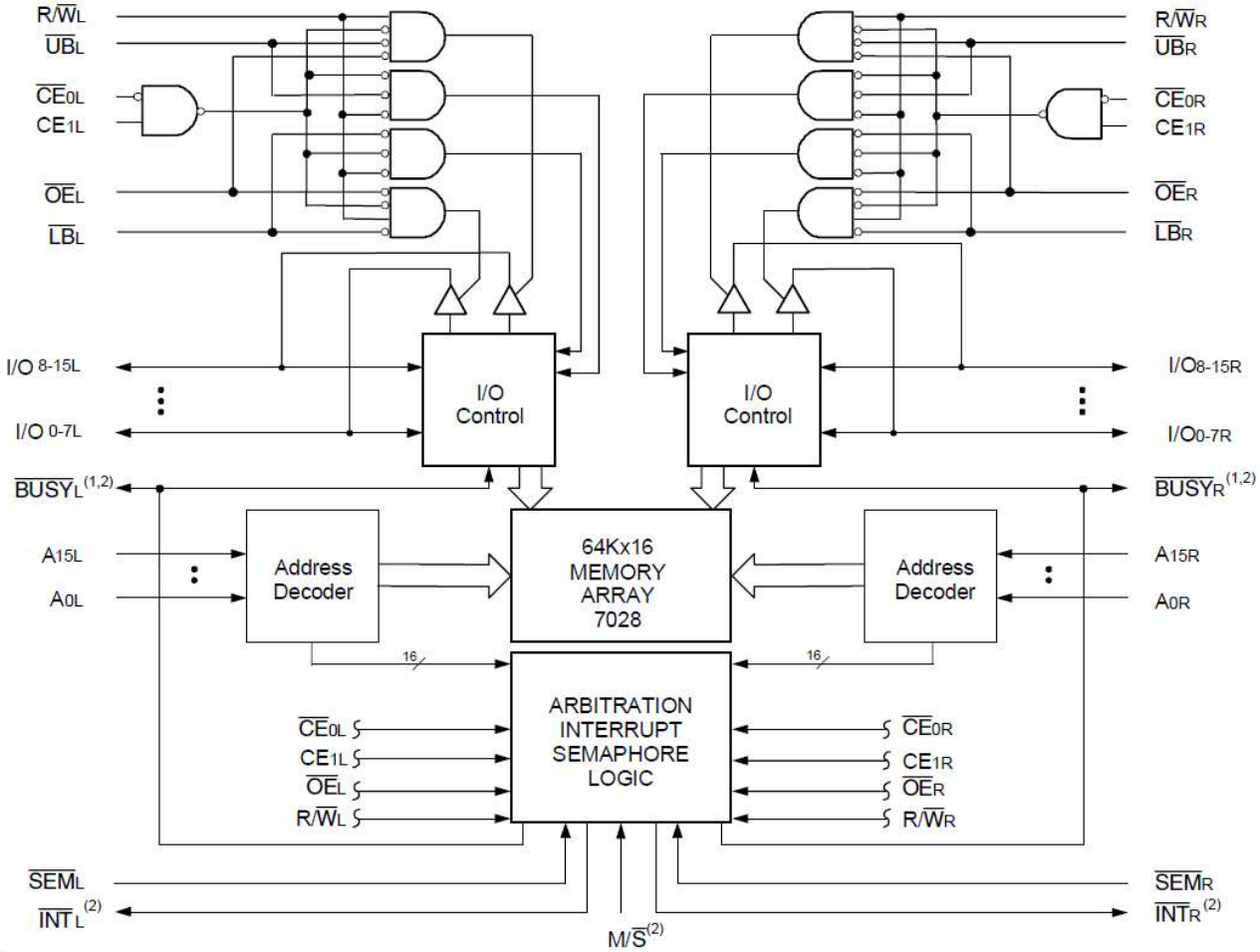
- Use extern keyword
- Kernel launch should provide the shared memory size
- You can only use 1D array
  - No problem. Just put a little more effort into index calculation
- Only one array
  - No problem. Just split the array into multiples with pointer arithmetic

CUDA MEMORY MODEL

- Shared memory can provide 32 concurrent reads/writes per cycle
  - For a warp
- How?



DUAL PORT MEMORY



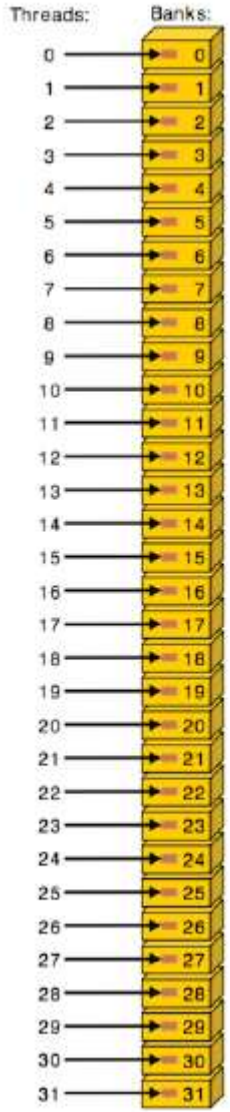
NOTES:
1. BUSY is an input as a Slave (M/S = VIL) and an output when it is a Master (M/S = VIH).
2. BUSY and INT are non-tri-state totem-pole outputs (push-pull).

- Alright, now a 32-port memory?

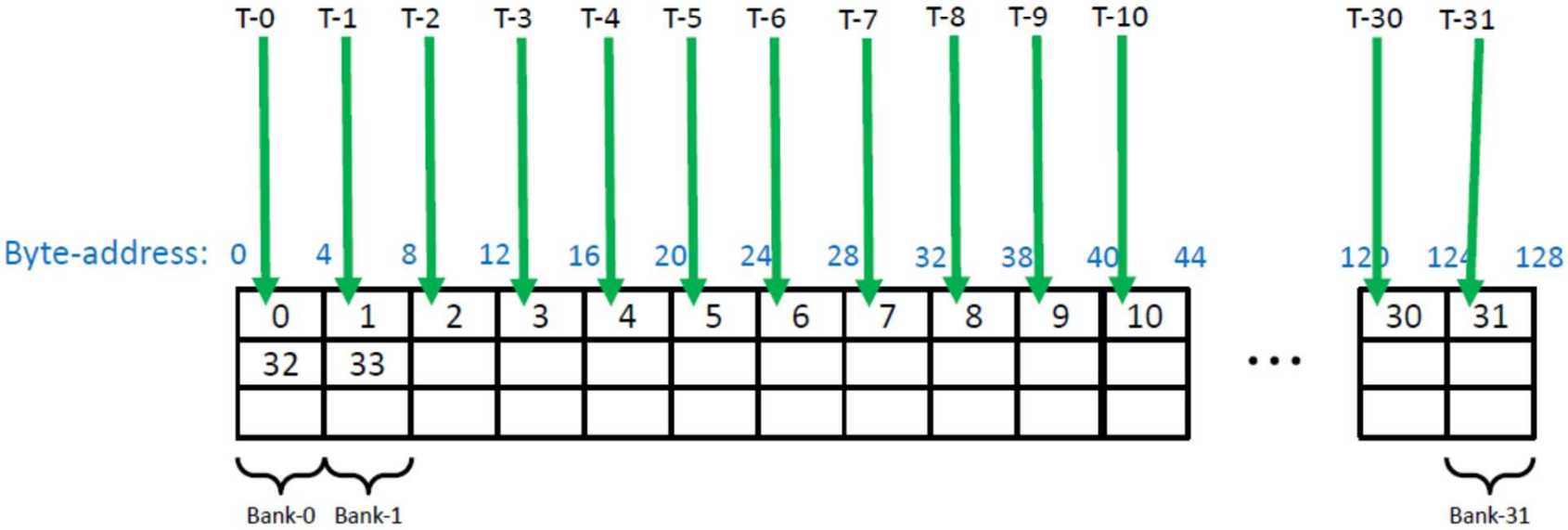
https://www.idt.com/sites/default/files/7028%20-%201%20-%20Block%20Diagram.png

MULTI-BANK ARCHITECTURE

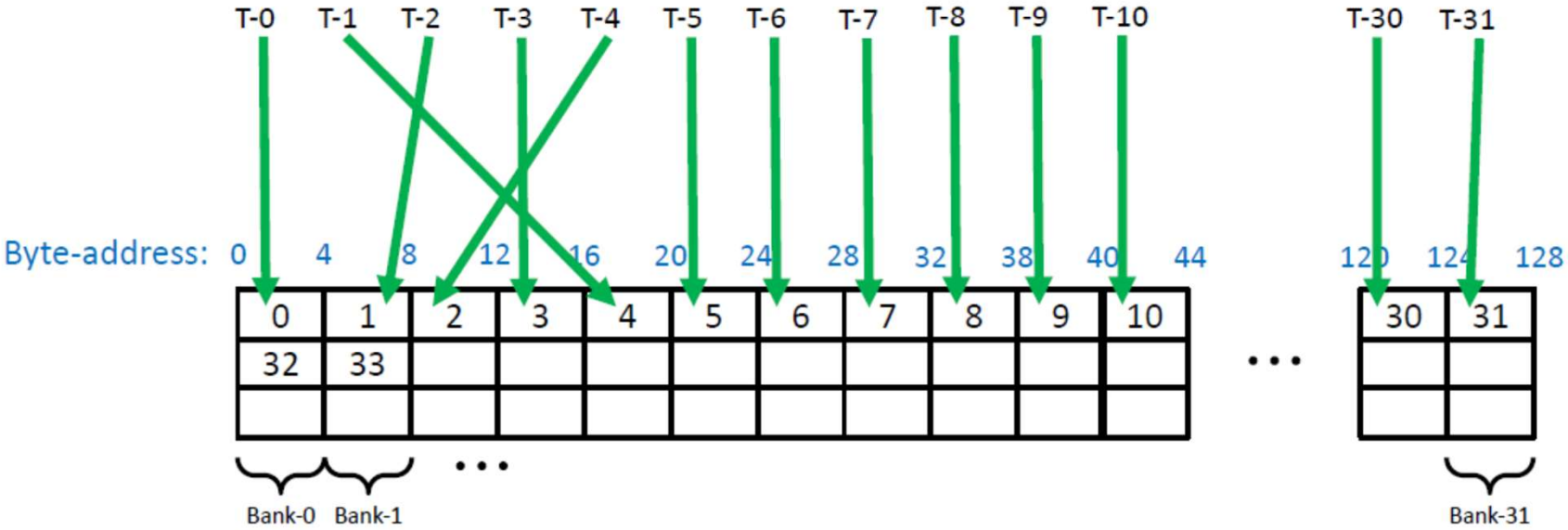
- Use 32 x single port memory
- Provide 32 accesses for 32 threads
- Each bank has 4B width



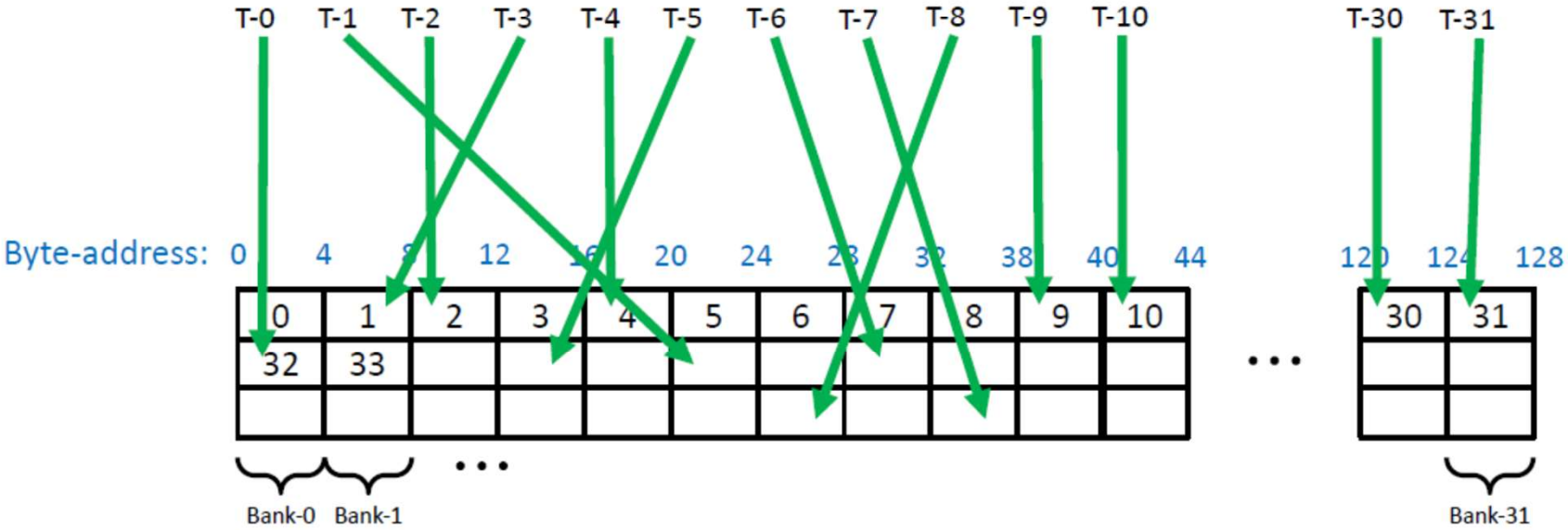
No Bank Conflicts



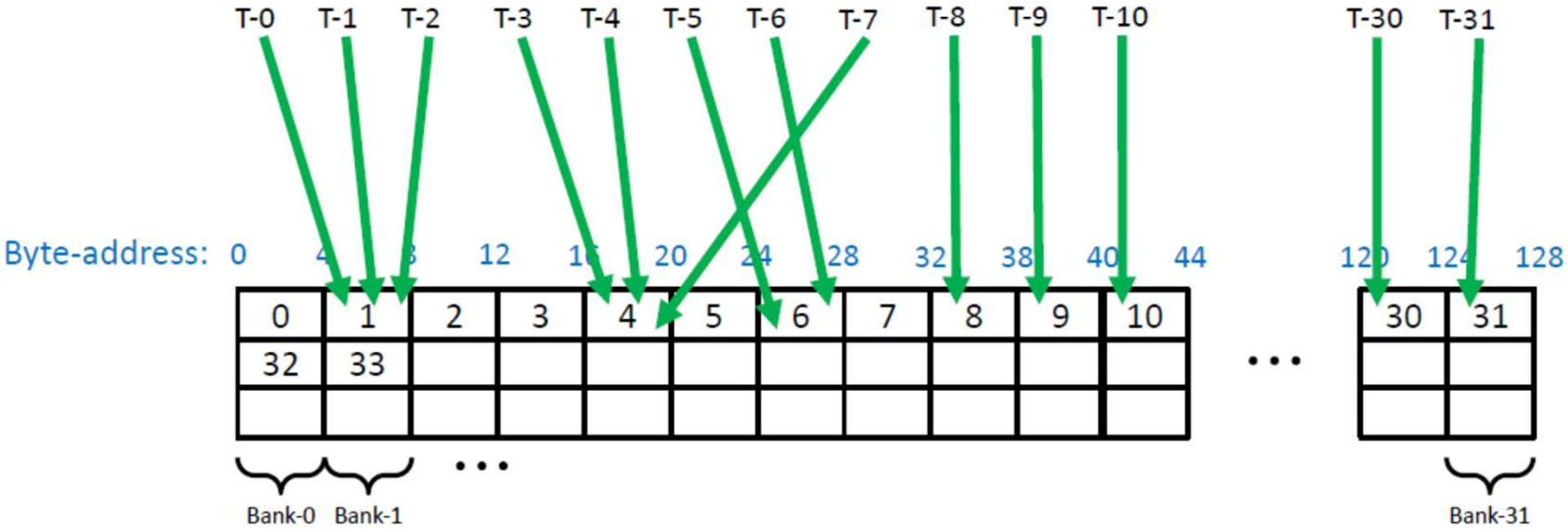
No Bank Conflicts



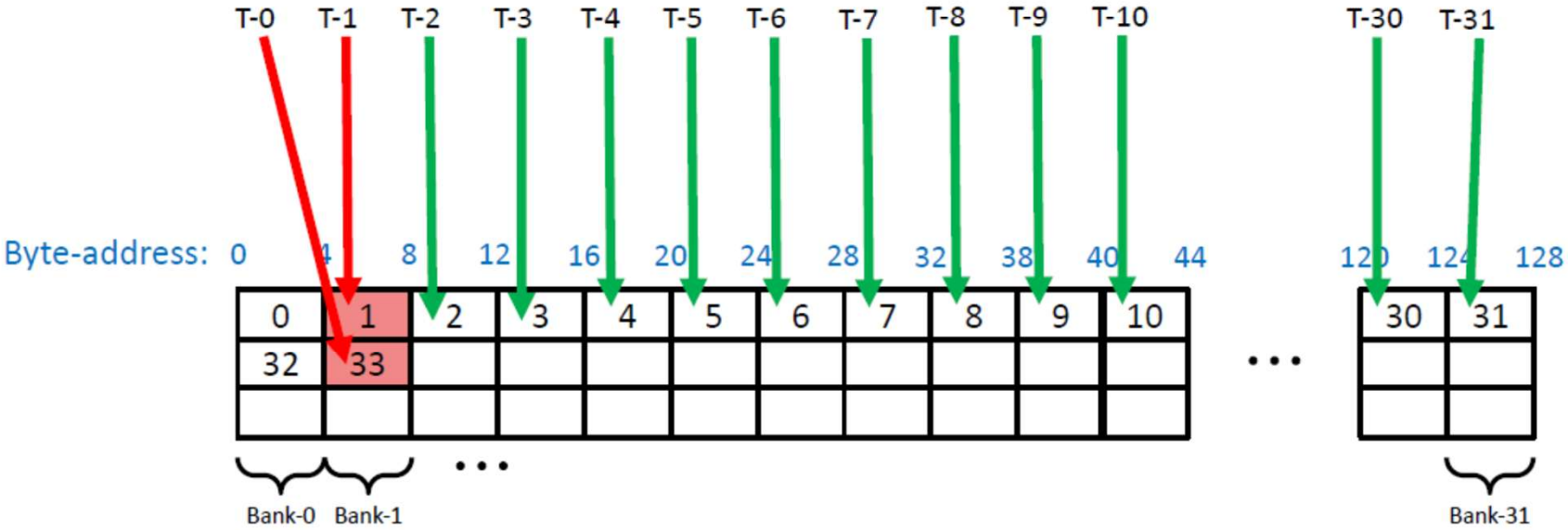
No Bank Conflicts



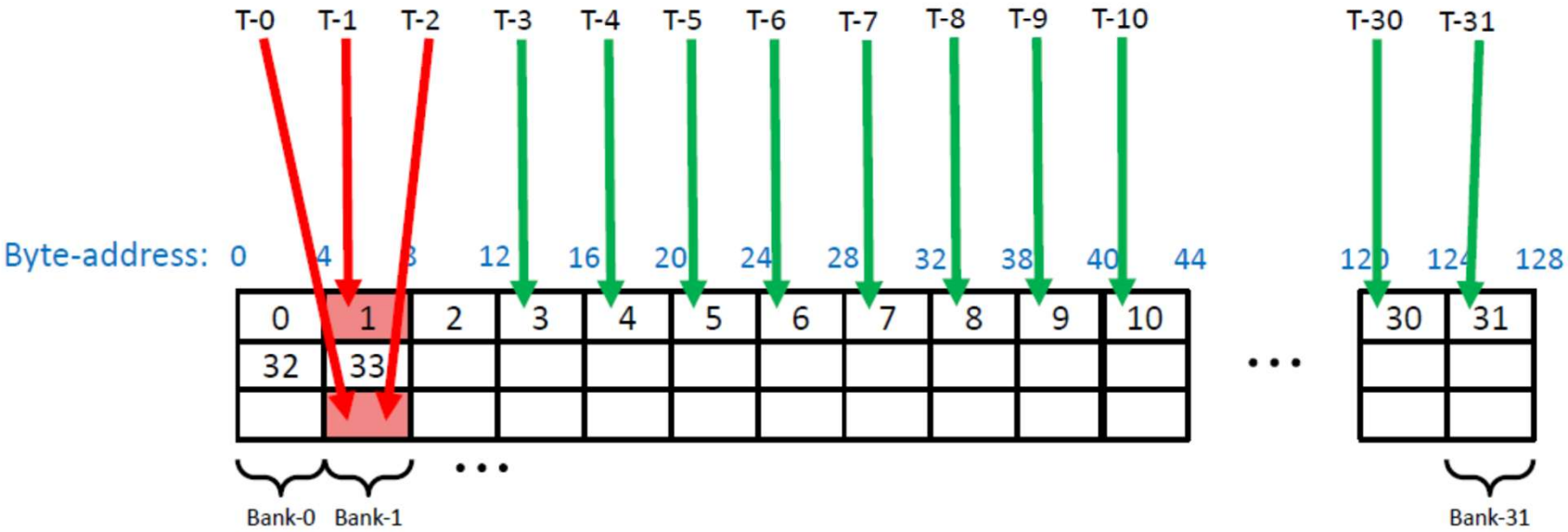
No Bank Conflicts



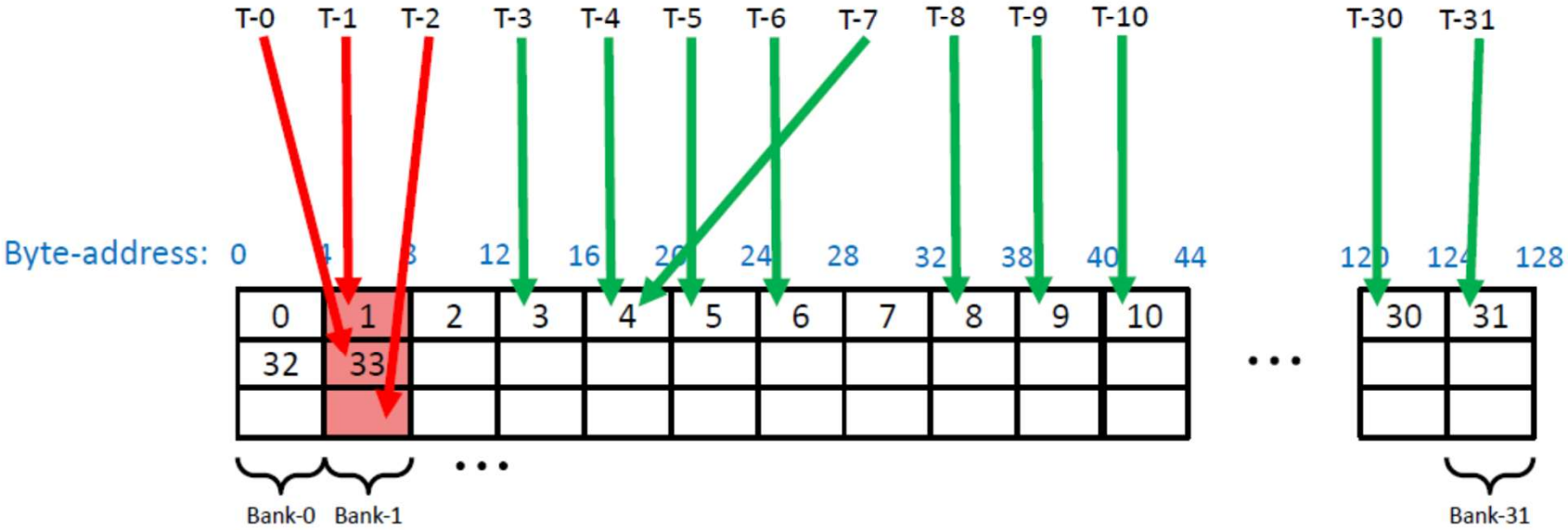
2-way Bank Conflict



2-way Bank Conflict



3-way Bank Conflict



MATRIX TRANSPOSE EXAMPLE

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

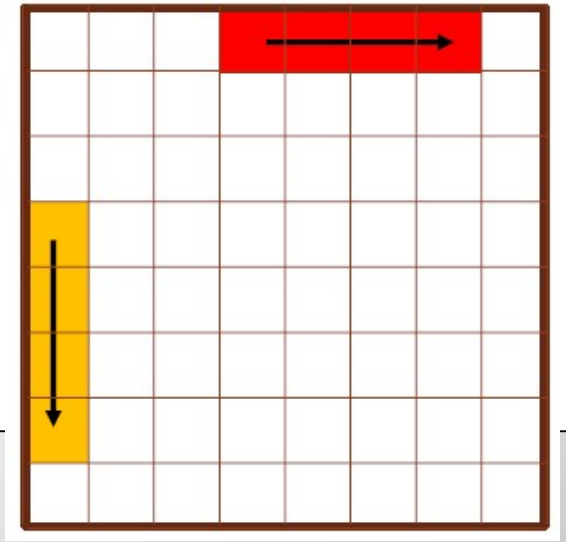
1,1	2,1	3,1	4,1
1,2	2,2	3,2	4,2
1,3	2,3	3,3	4,3
1,4	2,4	3,4	4,4

- Matrix transpose
- We have tried this, right?
- How would you program this using cuda?

TRY 0

```
  __global__
void transposeKernel_1D_0(float* A_d, float* B_d, long long int n)
{
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  int col = tid % n;
  int row = tid / n;

  //coalesced read
  if(row < n && col < n) {
    B_d[col*n + row] = A_d[row*n+col];
  }
}
```

32 reads



32 writes

- Read: coalesced
- Write: not coalesced

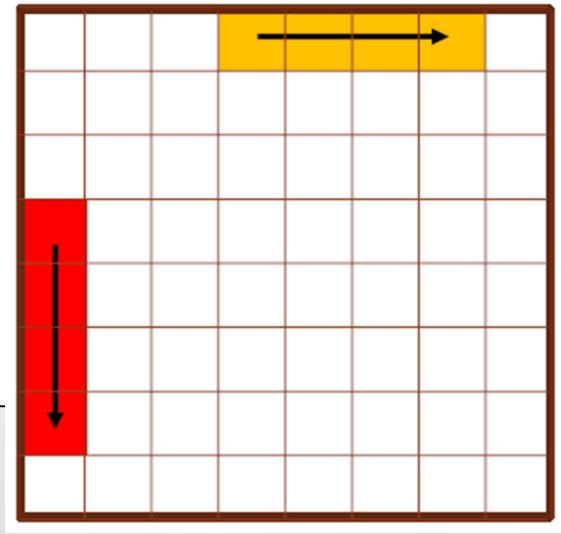
- 16384 x 16384 matrix
  - Kernel 0: 15.3sec
  - Kernel 1: 15.3sec

TRY 1

```
 __global__
void transposeKernel_1D_1(float* A_d, float* B_d, long long int n)
{
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  int col = tid % n;
  int row = tid / n;

  //coalesced write
  if(row < n && col < n) {
    B_d[row*n + col] = A_d[col*n+row];
  }
}
```

32 writes

32 reads



- Read: not coalesced
- Write: coalesced

- 16384 x 16384 matrix
  - Kernel 0: 15.3sec
  - Kernel 1: 15.3sec

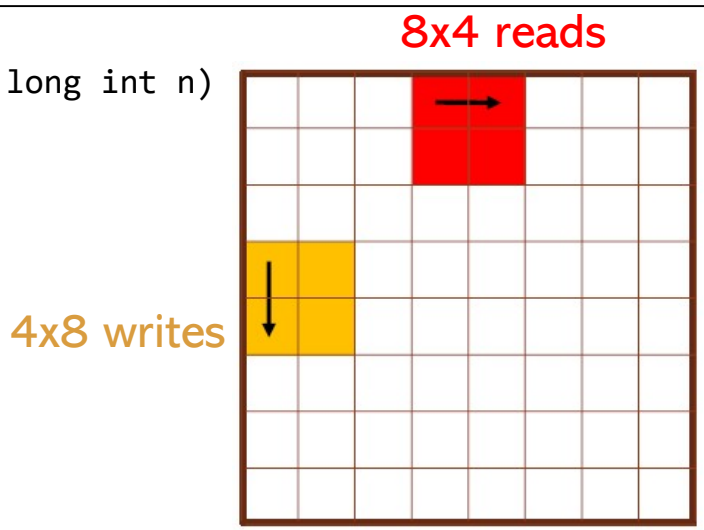
TRY 2,3

```
 __global__
void transposeKernel_2D_0(float* A_d, float* B_d, long long int n)
{
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if(row < n && col < n) {
    B_d[col*n + row] = A_d[row*n + col];
  }
}

  __global__
void transposeKernel_2D_1(float* A_d, float* B_d, long long int n)
{
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if(row < n && col < n) {
    B_d[row*n + col] = A_d[col*n + row];
  }
}
```

- Using 8x8 blocks, How are they being coalesced?

TRY 2,3

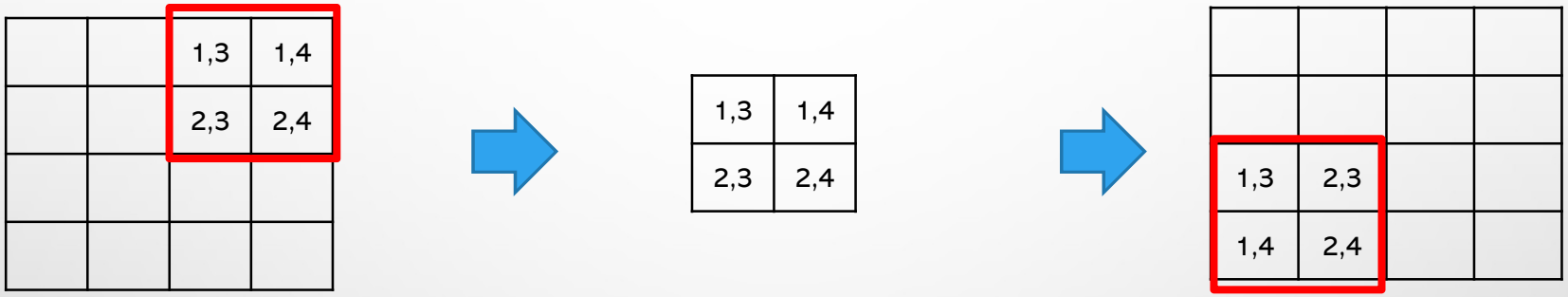
```
 __global__
void transposeKernel_2D_0(float* A_d, float* B_d, long long int n)
{
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if(row < n && col < n) {
    B_d[col*n + row] = A_d[row*n + col];
  }
}

  __global__
void transposeKernel_2D_1(float* A_d, float* B_d, long long int n)
{
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if(row < n && col < n) {
    B_d[row*n + col] = A_d[col*n + row];
  }
}
```



- Partially coalesced read
- Partially coalesced write

- 16384 x 16384 matrix
  - Kernel 0: 15.3sec
  - Kernel 1: 15.3sec
  - Kernel 2: 1.10sec
  - Kernel 3: 0.96sec

CORNER TURNING



- Use shared memory (corner turning)
- Read in a coalesced order to the shared memory
- Write in a coalesced order from the shared memory

TRY 4

```
 __global__
void transposeKernel_2D_SM(float* A_d, float* B_d, long long int n)
{
  extern __shared__ float buffer[];
  //__shared__ float buffer[1024];
  int blockRow = blockIdx.y;
  int blockCol = blockIdx.x;
  int localRow = threadIdx.y;
  int localCol = threadIdx.x;
  const int tileWidth = blockDim.x;

  //coalesced read
  int srcRow = blockRow*tileWidth + localRow;
  int srcCol = blockCol*tileWidth + localCol;
  if(srcRow < n && srcCol < n) {
    buffer[localRow*tileWidth + localCol] = A_d[srcRow*n + srcCol];
  }
  __syncthreads();

  //coalesced write
  //this transposes inter-block
  int dstRow = blockCol*tileWidth + localRow;
  int dstCol = blockRow*tileWidth + localCol;
  if(dstRow < n && dstCol < n) {
    //this transposed within block
    B_d[dstRow*n + dstCol] = buffer[localCol*tileWidth + localRow];
  }
}
```

- Use 32x32 blocks (for coalescing)

TRY 4

```
 __global__
void transposeKernel_2D_SM(float* A_d, float* B_d, long long int n)
{
  extern __shared__ float buffer[];
  //__shared__ float buffer[1024];
  int blockRow = blockIdx.y;
  int blockCol = blockIdx.x;
  int localRow = threadIdx.y;
  int localCol = threadIdx.x;
  const int tileWidth = blockDim.x;

  //coalesced read
  int srcRow = blockRow*tileWidth + localRow;
  int srcCol = blockCol*tileWidth + localCol;
  if(srcRow < n && srcCol < n) {
    buffer[localRow*tileWidth + localCol] = A_d[srcRow*n + srcCol];
  }
  __syncthreads();

  //coalesced write
  //this transposes inter-block
  int dstRow = blockCol*tileWidth + localRow;
  int dstCol = blockRow*tileWidth + localCol;
  if(dstRow < n && dstCol < n) {
    //this transposed within block
    B_d[dstRow*n + dstCol] = buffer[localCol*tileWidth + localRow];
  }
}
```

- Use 32x32 blocks (for coalescing)

- 16384 x 16384 matrix
  - Kernel 0: 15.3sec
  - Kernel 1: 15.3sec
  - Kernel 2: 1.10sec
  - Kernel 3: 0.96sec
  - Kernel 4: 0.96sec
    - What????

TRY 4

```
 __global__
void transposeKernel_2D_SM(float* A_d, float* B_d, long long int n)
{
  extern __shared__ float buffer[];
  //__shared__ float buffer[1024];
  int blockRow = blockIdx.y;
  int blockCol = blockIdx.x;
  int localRow = threadIdx.y;
  int localCol = threadIdx.x;
  const int tileWidth = blockDim.x;

  //coalesced read
  int srcRow = blockRow*tileWidth + localRow;
  int srcCol = blockCol*tileWidth + localCol;
  if(srcRow < n && srcCol < n) {
    buffer[localRow*tileWidth + localCol] = A_d[srcRow*n + srcCol]
  }
  __syncthreads();

  //coalesced write
  //this transposes inter-block
  int dstRow = blockCol*tileWidth + localRow;
  int dstCol = blockRow*tileWidth + localCol;
  if(dstRow < n && dstCol < n) {
    //this transposed within block
    B_d[dstRow*n + dstCol] = buffer[localCol*tileWidth + localRow];
  }
}
```

- Use 32x32 blocks (for coalescing)

B0	B1	B2	B3		B31
[0,0]	[0,1]	[0,2]	[0,3]		[0,31]
[1,0]	[1,1]	[1,2]	[1,3]		[1,31]
[2,0]	[2,1]	[2,2]	[2,3]		[2,31]
		…			…
[31,0]	[31,1]	[31,2]	[31,3]		[31,31]

- Stride of 32
  - 32-way conflict!
  - Kernel 4 is even slower on some platforms

BANK CONFLICT

B0	B1	B2	B3
[0,0]	[0,1]	[0,2]	[0,3]
[1,0]	[1,1]	[1,2]	[1,3]
[2,0]	[2,1]	[2,2]	[2,3]
	…		
[31,0]	[31,1]	[31,2]	[31,3]

B31
[0,31]
[1,31]
[2,31]
…
[31,31]

- Solution?
- We have seen this before, right?

  (x) Use 33x33 threadblock?

  (o) Use Padding

TRY 5

- Same code, but with padding
- No conflict :)

```
const int SKEW = 1;
  __global__
void transposeKernel_2D_SM_skew(float* A_d, float* B_d, long long int n)
{
  extern __shared__ float buffer[];
  //__shared__ float buffer[1024];
  int blockRow = blockIdx.y; int blockCol = blockIdx.x;
  int localRow = threadIdx.y; int localCol = threadIdx.x;
  const int tileWidth = blockDim.x;
  const int SMWidth = blockDim.x + SKEW;

  //coalesced read
  int srcRow = blockRow*tileWidth + localRow;
  int srcCol = blockCol*tileWidth + localCol;
  if(srcRow < n && srcCol < n) {
    buffer[localRow*SMWidth + localCol] = A_d[srcRow*n + srcCol];
  }
  __syncthreads();

  //coalesced write
  //this transposes inter-block
  int dstRow = blockCol*tileWidth + localRow;
  int dstCol = blockRow*tileWidth + localCol;
  if(dstRow < n && dstCol < n) {
    //this transposed within block
    B_d[dstRow*n + dstCol] = buffer[localCol*SMWidth + localRow];
  }
}
```

B0	B1	B2	B3	B31
[0,0]	[0,1]	[0,2]	[0,3]	[0,31]
[0,32]	[1,0]	[1,1]	[1,2]	[1,30]
[1,31]	[1,32]	[2,0]	[2,1]	[2,29]
		...		...

- 16384 x 16384 matrix
  - Kernel 0: 15.3sec
  - Kernel 1: 15.3sec
  - Kernel 2: 1.10sec
  - Kernel 3: 0.96sec
  - Kernel 4: 0.96sec
  - Kernel 5: 0.54sec

SUMMARY

- Shared memory and coalescing is usually the key to gaining performance

1. Load data into shared memory
2. Consider coalescing on loads
3. Avoid bank conflicts